

Fast and the Furious: Hot Starts in Pursuit-Evasion Games

Gabriel Smithline

University of Michigan

Ann Arbor, Michigan, United States

gsmithl@umich.edu

Scott Nivison

Air Force Research Laboratory

Shalimar, Florida, United States

scott.nivison.1@us.af.mil

ABSTRACT

Effectively positioning pursuers in pursuit-evasion games without prior knowledge of evader locations remains a significant challenge. A novel approach that combines game-theoretic control theory with Graph Neural Networks is introduced in this work. By conceptualizing pursuer configurations as strategic arrangements and representing them as graphs, a Graph Feature Space is constructed via multi-objective optimization to identify Pareto-optimal configurations. A Graph Convolutional Network (GCN) is trained on these Pareto-optimal graphs to generate strategically effective initial configurations, termed *hot starts*. Empirical evaluations demonstrate that the GCN-generated hot starts provide a significant advantage over random configurations. In scenarios considering multiple pursuers and evaders, this method hastens the decline in evader survival rates, reduces pursuer travel distances, and enhances containment, showcasing clear strategic benefits.

KEYWORDS

Differential Game Theory; Pursuit-Evasion Games; Graphical Neural Networks; Swarm Robotics

ACM Reference Format:

Gabriel Smithline and Scott Nivison. 2025. Fast and the Furious: Hot Starts in Pursuit-Evasion Games. In *Proc. of the 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2025)*, Detroit, Michigan, USA, May 19 – 23, 2025, IFAAMAS, 9 pages.

1 INTRODUCTION

Pursuit-evasion games model interactions between a set of pursuers and evaders in differential games. These games are critically important in fields such as control, aerospace, and robotics, and form the foundation of adversarial optimal control [15, 35]. Since their inception in the 1950s and 1960s, pursuit-evasion games have seen significant advancements, with one of the most foundational contributions being the Hamilton-Jacobi-Isaacs (HJI) equation [15] introduced by Rufus Isaacs. The HJI equation, a generalization of the Hamilton-Jacobi-Bellman (HJB) equation [4], extends to zero-sum games where one player aims to minimize a cost function while another aims to maximize it. In these games, the optimal strategies for pursuers and evaders must satisfy the HJI. The equation, a partial differential equation (PDE), captures the game’s dynamics, including player positions, velocities, and optimal control actions. The solution to the HJI equation, known as the value function, represents the minimum cost for pursuers to capture evaders, considering the evaders’ optimal responses.

Despite these advancements, many open-ended questions remain. A significant challenge is strategically placing pursuers without knowing the number or locations of evaders. Developing methods to address this issue could enhance algorithm effectiveness and efficiency, thereby reducing collective pursuer costs.

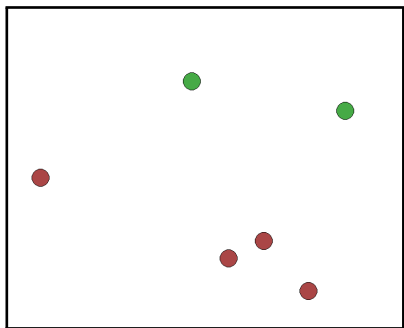
In this paper, a novel approach is proposed that leverages artificial intelligence and machine learning techniques to offline generate initial configurations for pursuers before evaders appear, a strategy termed *hot starts*. Inspired by recent advances in Graph Neural Networks (GNNs) [31], particularly the methodology described in Google DeepMind’s TacticAI developed for Liverpool FC [34], we aim to apply similar methods to pursuit-evasion games. Two pure pursuit game scenarios are considered: One Evader vs. Many Pursuers and Many Evaders vs. Many Pursuers.

Our method demonstrates enhancements by not making direct assumptions about evader positions, quantities, algorithms, optimization issues, and state-space simplifications with these hot starts. To overcome the lack of domain-specific expert feedback on configurations, games are simulated using the Multi-Agent Particle Environment (MPE) [24, 27], a flexible simulation framework designed for multi-agent interactions (see Figure 3). Through these simulations, empirical data is generated, which serves two purposes: first, it provides the data needed to train the Graph Convolutional Network (GCN) [20] to extrapolate patterns; second, it allows us to construct a Graph Feature Space (GFS) using three features: U_{capture} (capture potential), U_{distance} (distance), and U_{heading} (heading angle). Survival analysis and log-rank tests are performed to empirically evaluate whether hot starts have a statistically significant impact. Key contributions are outlined as follows:

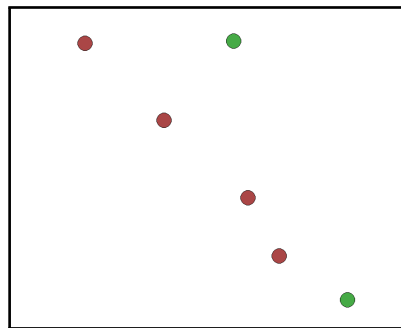
- Encode pursuer configurations in the GFS, allowing us to solve a multi-objective optimization problem offline and construct a Pareto frontier of pursuer configurations.
- Train the GCN using a loss function designed to learn from the Pareto-valued configurations. Track hypervolume and generational distance metrics to measure how the GCN learns the Pareto front and how well it generalizes by measuring the volume of the frontier it covers.
- Inspired by TacticAI, generate numerous configurations with a GCN and use statistical methods to evaluate the impact of these configurations on pursuer efficiency and evader capture.

2 RELATED WORK

To model effective gameplay in pursuit-evasion games, algorithms often utilize Apollonius circles [11, 15, 29], and strategies based on Voronoi diagrams or Delaunay triangulation [3, 33] (for a more thorough overview on differential games, see Appendix Section A). Previous works have explored simplified, discretized environments



(a) 4 Pursuers vs. 2 Evaders Random Start



(b) 4 Pursuers vs. 2 Evaders Hot Start

Figure 1: Images of a hot start and a random start in 4 Pursuer vs. 2 Evader Game (pursuers are in red and evaders are in green).

where pursuers are explicitly assigned to evaders based on probabilistic estimation techniques, or where the explicit algorithms of the adversary are known by each party [2, 12, 19, 37]. Other studies highlight redundant allocation of pursuers [33], effectively increasing the cost of pursuit for the collective of pursuers.

Recent works have extended the application of graph-based learning techniques to multi-agent systems in discrete environments. For instance, Li et al. [21, 22] structured agent interactions within a graph to solve pursuit-evasion games, with a focus on real-time coordination and task assignment. While effective for discrete settings, these methods do not address the dynamic and continuous aspects of real-world environments, nor do they tackle the same problem focused on in this work. This work diverges by introducing a novel process for generating "hot starts" without prior knowledge of evader positions. By leveraging differential game theory and operating in continuous space, time-dependent algorithms are accounted for, for both pursuers and evaders, which contrasts with the real-time updates used in discrete environments. This multi-objective optimization approach allows us to generate Pareto-optimal configurations that improve pursuer efficiency from the outset. The combination of continuous space, differential dynamics, and the distinct focus on initial agent allocation sets this work apart in the domain of pursuit-evasion games, making it applicable to a wider range of real-world scenarios.

The concept of initial agent allocation in large-scale systems has also been explored in particle swarm optimization and genetic algorithms, particularly in population initialization, where the goal is to initialize the population advantageously [17]. Traditionally, evolutionary algorithms use pseudo-random number generators for population initialization; more recent work explores intelligent initialization through AI-based heuristics [18].

While GNN-based approaches like those used in Google DeepMind’s TacticAI for sports strategy [34] have shown success in generating player configurations for specific tasks (e.g., corner kicks), applying similar methods to pursuit-evasion games poses distinct challenges. In sports, configurations benefit from domain-specific expertise and clear objectives, such as the positioning of players based on real-time feedback from coaches. By contrast, pursuit-evasion games often lack this level of expert guidance, and there is the added complexity of not knowing evader positions or quantities in advance.

Previous works have not fully explored the challenge of strategically positioning pursuers without such prior knowledge, which is a fundamental aspect of real-world pursuit-evasion scenarios. This approach seeks to explore this gap by leveraging GNNs to generate initial configurations, termed "hot starts," without making direct assumptions about the number or location of evaders. This method has potential applications in a wide range of domains, including UAV search and rescue [9], wildlife protection [7, 36], human-computer interaction [23], traffic management [25], and general security [1, 32], where such strategic initialization could significantly enhance operational efficiency.

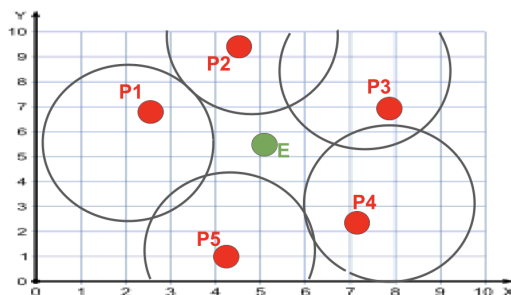


Figure 2: Image of 5 pursuers (in red) surrounding an evader (in green), illustrating cooperative containment. Black circles represent capture radii.

3 CONTROL ALGORITHMS

3.1 1 Evader vs. Many Pursuers

Control algorithms using heuristics inspired by strategies from [10, 11] were adapted for our scenarios, taking into account the varying speed ratios between evaders and pursuers. In the literature, different speed ratios lead to distinct geometric constructs: Apollonius circles or Cartesian ovals when $\gamma \neq 1$, and perpendicular bisectors or hyperbolas when $\gamma = 1$. In our work, for the 1 evader vs. many pursuers scenario, we assume $\gamma = \frac{v_E}{v_P} > 1$, where the evader is faster than the pursuers. Here, the evader identifies the weakest link between pursuers to escape, while pursuers aim to contain the evader strategically. (See Appendix B for detailed algorithms and system dynamics.)

3.2 Many Evaders vs. Many Pursuers

For many-vs-many scenarios ($\gamma = \frac{1}{2} < 1$), where pursuers are faster than evaders, we employ a hybrid approach combining geometry-based controls with PDE solutions. Specifically, we use a first-order upwind finite difference scheme with directional bias: backward differences are selected for negative gradients and forward differences for positive gradients. This scheme provides numerical stability while respecting the information flow direction in the HJI equation, consistent with methods described in [26, 28]. The Hamiltonian computation explicitly accounts for the minimizing pursuer and maximizing evader roles, ensuring appropriate upwind direction selection based on agent type.

Our hybrid approach uses a weighted parameter $\alpha = 0.5$ to combine geometric distance metrics with PDE value function information in the assignment cost matrix for the Hungarian algorithm. The geometric measures specifically include: (1) Euclidean distances between pursuers and evaders for the assignment cost matrix, (2) normalized direction vectors for determining movement trajectories, (3) relative bearing angles for optimal interception paths, and (4) proximity-based avoidance vectors that allow evaders to compute escape directions from nearby pursuers. Pursuers are allocated to evaders by minimizing this hybrid cost, and control actions are computed using both gradient information from the PDE and direct geometric vectors, combining them as:

$$\mathbf{u}_{\text{final}} = \alpha \cdot \mathbf{u}_{\text{geometric}} + (1 - \alpha) \cdot \mathbf{u}_{\text{gradient}} \quad (1)$$

where $\mathbf{u}_{\text{geometric}}$ is derived from the instantaneous geometric relationship between agents, and $\mathbf{u}_{\text{gradient}}$ comes from the spatial derivatives of the value function. Evaders employ escape strategies based on gradient ascent of the value function combined with geometric avoidance of nearby pursuers, computing avoidance vectors that are inversely proportional to the squared distance from approaching pursuers when they enter a predefined avoidance radius.

Our numerical approach, while using discrete time steps, aims to approximate continuous-time game dynamics. The qualitative results align with theoretical predictions for pursuit-evasion scenarios with the given speed ratios [5, 14, 35], demonstrating the efficacy of our implementation despite the inherent limitations of numerical approximation.

4 GRAPH FEATURE SPACE

4.1 Graph Features

The Graph Feature Space (GFS) is an objective space where configurations of agents are mapped based on certain features, sampled during our data-generation phase. The three features we focus on to construct the Pareto frontier are: U_{capture} (capture potential), U_{distance} (distance), and U_{heading} (heading angle). These features are computed offline for each pursuer within the PettingZoo environment, as illustrated in Figure 3. We delve into their definitions below.

4.2 Pareto Frontier Definition

The Pareto frontier is formally defined as the subset $\Pi_p \subset \Pi$, where Π is the set of all possible configurations. Formally, Π_p is found such that:

$$\Pi_p = \{G_i \in \Pi \mid \nexists G_j \in \Pi \text{ such that } \hat{c}_{G_j} \succ \hat{c}_{G_i}\}, \quad (2)$$

where $\hat{c}_{G_i}$ is the vector containing our three features of the GFS, for configuration G_i , and $\hat{c}_{G_j} \succ \hat{c}_{G_i}$ indicates that the feature vector of G_j strictly dominates that of G_i . More precisely, for G_j to strictly dominate G_i :

$$\hat{c}_{G_j} \succ \hat{c}_{G_i} \iff \forall k \in \{1, \dots, n\}, \hat{c}_{G_j,k} \geq \hat{c}_{G_i,k} \wedge \exists l \in \{1, \dots, n\}, \hat{c}_{G_j,l} > \hat{c}_{G_i,l} \quad (3)$$

where n is the number of objectives. The set Π_p thus represents the Pareto-optimal configurations, where no configuration in Π strictly dominates any element of Π_p [38].

4.3 Feature Definitions

Specifically, U_{capture} is expressed as:

$$U_{\text{capture}} = 2 \left(1 - \frac{1}{1 + e^{\frac{-d_i + r}{r}}} \right) \quad (4)$$

where:

- d_i is the distance between the pursuer and the i -th evader.
- r is the capture radius.
- n is the number of evaders.

This utility is designed to be 1 when the distance $d_i = r$, and approaches 0 as the distance d_i increases. When $d_i < r$, $U_{\text{capture}} > 1$, indicating a higher capture potential within the capture radius. The sigmoid component smoothly transitions the utility value, avoiding abrupt changes and adding stability to optimization and decision-making processes.

U_{distance} is expressed as:

$$U_{\text{distance}} = \min_{i=1, \dots, n} \|\mathbf{p} - \mathbf{e}_i\| \quad (5)$$

where:

- $\mathbf{p}$ is the position vector of the pursuer.
- $\mathbf{e}_i$ is the position vector of the i -th evader.
- n is the number of evaders.

This utility represents the shortest distance between the pursuer and any of the evaders. It focuses on the closest evader, which is critical in pursuit-evasion games where the goal is to minimize the distance to capture the nearest evader.

Finally, U_{heading} is expressed as:

$$U_{\text{heading}} = \frac{1}{n} \sum_{i=1}^n \arccos \left(\frac{\mathbf{v}_p \cdot (\mathbf{e}_i - \mathbf{p})}{\|\mathbf{v}_p\| \|\mathbf{e}_i - \mathbf{p}\| + \epsilon} \right) \quad (6)$$

where:

- $\mathbf{v}_p$ is the velocity vector of the pursuer.
- $\mathbf{p}$ is the position vector of the pursuer.
- $\mathbf{e}_i$ is the position vector of the i -th evader.
- n is the number of evaders.
- $\epsilon = 10^{-9}$ is a small constant to prevent division by zero.

This utility measures the alignment between the pursuer's heading direction and the directions to each evader. Lower values of U_{heading} (i.e., smaller angles) indicate better alignment between a pursuer's heading and the direction to an evader when locations are known. In scenarios where evader locations are unknown, maximizing U_{heading} (achieved by averaging) encourages pursuers to consider a wider range of heading angles, increasing the likelihood that pursuers are heading in beneficial directions when evaders appear.

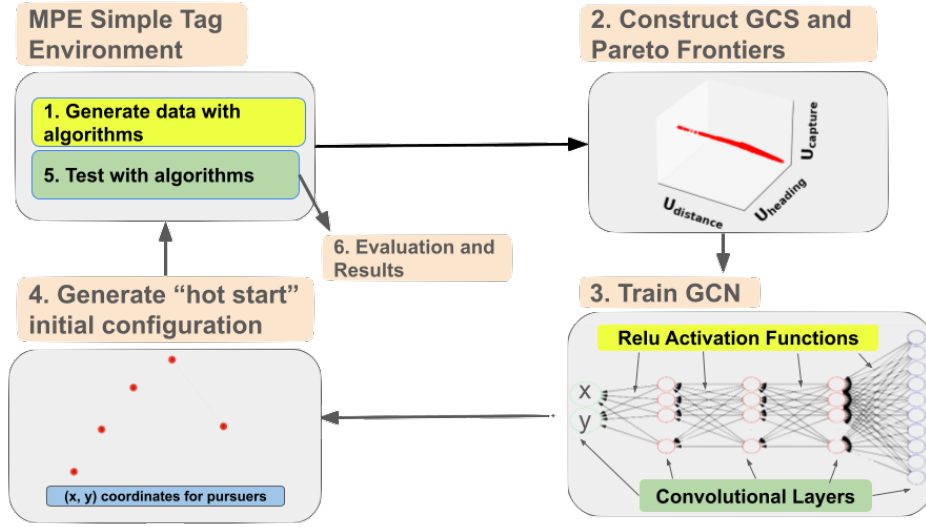


Figure 3: Pipeline diagram of the Pursuit-Evasion Games framework. Starting from the MPE Simple Tag Environment, the process follows these steps: 1. Generate data using control algorithms; 2. Construct the Graph Feature Space (GFS) and identify Pareto frontiers; 3. Train the Graph Convolutional Network (GCN); 4. Generate Hot Starts (each red dot represents x and y coordinates); 5. Apply these Hot Starts to the control algorithm; and 6. Perform statistical evaluation.

Illustrative examples demonstrating the functionality of U_{heading} and U_{capture} are provided in Section D of the Appendix.

4.4 Normalization and Aggregation

To ensure comparability among the different features and maintain the sum-to-one constraint, we normalize the aggregated features proportionally. Specifically, for each configuration G_i , we aggregate the utilities by averaging over all pursuers:

$$U_k(G_i) = \frac{1}{N_p} \sum_{j=1}^{N_p} U_k^{(j)}, \quad k \in \{\text{capture, distance, heading}\}$$

where N_p is the number of pursuers in the configuration G_i , and $U_k^{(j)}$ is the feature for pursuer j .

To enforce the sum-to-one constraint across all utilities, each aggregated utility $U_k(G_i)$ is normalized by the total sum of all three utilities for the configuration:

$$\tilde{U}_k(G_i) = \frac{U_k(G_i)}{\sum_{m=1}^3 U_m(G_i)}$$

This ensures that:

$$\sum_{k=1}^3 \tilde{U}_k(G_i) = 1$$

By applying this proportional normalization, we maintain a balanced contribution of each utility feature in the multi-objective optimization process. This facilitates a fair and consistent comparison of pursuer configurations, ensuring that no single feature disproportionately influences the optimization outcomes.

4.5 Construction of the Graph Feature Space (GFS)

To construct the GFS and identify Pareto-optimal pursuer configurations, we formulate a constrained multi-objective optimization problem. We utilize the NSGA-II genetic algorithm [8], with a population size of 500 and 200 generations, to solve this problem. We start by generating a diverse set of pursuer configurations. Each configuration G_i consists of multiple pursuers positioned in the environment. These configurations are generated through simulations or sampling methods and are stored in the set Π .

For each configuration $G_i \in \Pi$, we compute the three normalized and aggregated features:

$$\tilde{U}_k(G_i) = \frac{U_k(G_i)}{\sum_{m=1}^3 U_m(G_i)}, \quad k \in \{\text{capture, distance, heading}\}$$

Our goal is to identify configurations that offer the best trade-offs among the three features. We formulate the multi-objective optimization problem as:

$$\underset{G_i \in \Pi}{\text{minimize}} \quad \mathbf{f}(G_i) = [-\tilde{U}_{\text{capture}}(G_i), \tilde{U}_{\text{distance}}(G_i), -\tilde{U}_{\text{heading}}(G_i)] \quad (7)$$

The objectives are:

- **Maximize** $\tilde{U}_{\text{capture}}(G_i)$: Increase the potential effectiveness in capturing evaders.
- **Minimize** $\tilde{U}_{\text{distance}}(G_i)$: Reduce the average minimum distance to evaders.
- **Maximize** $\tilde{U}_{\text{heading}}(G_i)$: Encourage pursuers to have diverse heading directions.

Since standard optimization algorithms perform minimization, we negate the objectives we wish to maximize. The proportional normalization ensures that each feature is appropriately scaled, facilitating a balanced optimization process.

From the Pareto-optimal configurations determined by our optimization problem, we construct graphs by adding edges between pursuer nodes called PP Edges.

4.5.1 Pursuer-Pursuer (PP) Edges. A PP edge exists between two pursuers if the distance between them is less than or equal to the average distance between all pairs of pursuers in the configuration. The weight of a PP edge is defined as:

$$w_{pp} = \frac{1}{d_{pp}} \quad (8)$$

where d_{pp} is the distance between the two pursuers.

Each node in the graph consists of the following features: $\tilde{U}_{\text{capture}}$, $\tilde{U}_{\text{distance}}$, $\tilde{U}_{\text{heading}}$, capture radius, x & y velocities, and x & y positions.

We employ the NSGA-II algorithm to solve this optimization problem. The algorithm operates as follows:

- (1) **Decision Variables:** The configurations G_i in Π .
- (2) **Evaluation:** For each configuration G_i , compute the normalized features and evaluate the objective vector $\mathbf{f}(G_i)$.
- (3) **Constraint Handling:** Enforce the constraint that the sum of the features equals 1.
- (4) **Selection:** Identify non-dominated configurations to form the Pareto front.

The result of the optimization is a set of Pareto-optimal configurations $\Pi_p \subset \Pi$. These configurations represent the best trade-offs among the objectives, where no other configuration in Π is strictly better in all objectives.

5 GRAPH CONVOLUTIONAL NETWORK

To generate our hot starts, we use a Graph Convolutional Network (GCN) trained on the graphs from the GFS that fall on the Pareto frontier. GCNs are advantageous for operating on graph-structured data, making them suitable for tasks like node classification, link prediction, and graph generation. The GCN follows a structure where it processes the features of pursuer nodes and their connections to output optimized configurations.

5.1 Pareto Loss

During training, we monitor several performance measures such as Hypervolume, Generational Distance (GD), Generational Distance Plus (GD+), Inverted Generational Distance (IGD), and Inverted Generational Distance Plus (IGD+) [6, 39]. Our primary loss function, called Pareto loss, is defined to measure the distance between the utility feature vector of the generated pursuer configuration and the utility feature vectors of the configurations on the Pareto front:

$$\mathcal{L}_{\text{Pareto}} = \min_{j \in \{1, \dots, m\}} \|\mathbf{y} - \mathbf{p}_j\| \quad (9)$$

where:

- $\mathbf{y}$ is the aggregated utility feature vector for the generated pursuer configuration, computed by averaging the utilities of all pursuers in the configuration:

$$\mathbf{y} = \frac{1}{n} \sum_{i=1}^n \left(\tilde{U}_{\text{distance}_i}, \tilde{U}_{\text{capture}_i}, \tilde{U}_{\text{heading}_i} \right)$$

where n is the number of pursuers.

- $\mathbf{p}_j$ is the utility feature vector of the j -th configuration on the Pareto front.
- m is the number of configurations on the Pareto front.

This loss function encourages the network to generate configurations whose aggregated utility vectors are close to those of the Pareto-optimal configurations.

5.2 Network Layers and Training

Our GCN processes the features of all pursuers in a configuration simultaneously, considering the interactions between pursuers through the graph structure. The input features for each pursuer node include:

- **Pursuer Position** (x, y): The initial positions of the pursuers.
- **Pursuer Velocity** (v_x, v_y): The initial velocities or headings of the pursuers.
- **Additional Features:** Such as initial utility estimates and capture radius or apollonious circle.

The edges in the graph represent connections between pursuers (PP edges), allowing the GCN to aggregate information from neighboring nodes and consider their relative positions and features.

The network outputs adjusted positions and velocities for all pursuers simultaneously, ensuring that the generated configuration is coherent and optimized at the group level. By processing the entire configuration as a graph, the GCN accounts for interactions between pursuers, preventing issues such as overlapping positions and promoting effective spatial distribution.

Our network architecture and training parameters are detailed in Table 1. Training took place on 4,700 graphs from 12 game types (combinations of 1 to 5 evaders and 2 to 5 pursuers), effectively stacking the Pareto fronts of each game type to form the GFS. GD, GD+, IGD, and IGD+ were measured during training to assess how the generated graphs relate to our Pareto values. The hypervolume of the generated graphs was approximately 0.634 (see Appendix D). Hypervolume was not included in the loss function due to computational expense.

Although training ran for 150 epochs, the network converged earlier, as loss values decreased significantly over time. Each utility feature ranges from 0 to 1; thus, small changes in the GFS led to larger changes in actual results. Despite early convergence, extended training was necessary to fine-tune the network, as it was found that small changes in the GFS led to larger changes in actual results.

Table 2: Survival Probabilities for 4 Pursuers vs. 2 Evaders (GCN vs. Random).

Time	GCN	Rand	Time	GCN	Rand
0	1.0000	1.0000	21	0.8722	0.9133
1	0.9991	0.9984	22	0.8619	0.9061
2	0.9981	0.9958	23	0.8511	0.8984
3	0.9967	0.9931	24	0.8398	0.8903
4	0.9943	0.9902	25	0.8279	0.8817
5	0.9909	0.9873	26	0.8154	0.8725
6	0.9865	0.9842	27	0.8022	0.8627
7	0.9812	0.9811	28	0.7881	0.8522
8	0.9753	0.9777	29	0.7732	0.8409
9	0.9691	0.9742	30	0.7572	0.8285
10	0.9626	0.9705	31	0.7398	0.8149
11	0.9559	0.9667	32	0.7210	0.7998
12	0.9488	0.9626	33	0.7002	0.7829
13	0.9415	0.9583	34	0.6772	0.7637
14	0.9339	0.9537	35	0.6511	0.7418
15	0.9260	0.9489	36	0.6208	0.7160
16	0.9179	0.9438	37	0.5843	0.6848
17	0.9094	0.9384	38	0.5385	0.6448
18	0.9007	0.9327	39	0.4753	0.5882
19	0.8915	0.9266	40	0.3635	0.4846
20	0.8821	0.9201	—	—	—

Note: Times 0–20 are in the left half, 21–40 in the right half. Survival probabilities estimated via Kaplan-Meier at each discrete time step.

Table 1: Network Architecture and Loss Values, Training Parameters

(a) Network Architecture and Loss Values			(b) Training Parameters	
Layer	Nodes	Act.	Parameter	Value
Input	10	ReLU	Batch Size	1024
Hidden 1	64	ReLU	Learning Rate	1×10^{-4}
Hidden 2	32	ReLU	Optimizer	Adam
Hidden 3	64	ReLU	Weight Decay	1×10^{-2}
Output	2	Linear	Epochs	150
			Hardware	L4 GPU
Epoch	Loss			
1	0.145			
75	0.0247			
150	0.0234			

6 EXPERIMENTS AND DISCUSSION

After training, our network generated 1000 configurations for each game type. These configurations were generated offline without prior knowledge of evader locations or numbers, then stored in a

CSV, and supplied as initial pursuer locations as opposed to randomly placing them on a plane following a uniform distribution. To query our network, we passed in the amount of pursuers we would like it to generate configurations for (this could be thought of as the game type), as well as random input for the other features, meant to signify noise; this is meant to capture that fact that we have no observability about their velocities or locations. The network relies solely on pursuer-related inputs, producing positions and headings that are likely to perform well against unknown evader placements. This approach is distinct and fundamentally unique, as it enables the generation of strategically advantageous pursuer configurations without requiring real-time evader information.

We randomly sampled 10 batches of 100 configurations for each game type and tested them, comparing results with configurations generated using random placements. The results demonstrate that our network effectively learns generalizable configurations that improve performance in pursuit-evasion games, even without specific evader information.

6.1 Methodology

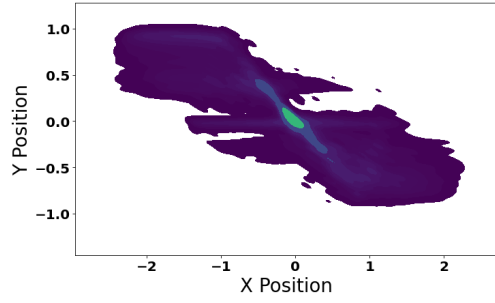
Hot starts are evaluated using a Monte Carlo analysis, aggregating results across numerous simulations. One can think of random starts as a control group and hot starts as the treatment group. Then the Kaplan-Meier estimator, a nonparametric metric from biostatistics [13, 16, 30], was used to estimate evader survival probabilities over time. Log-rank tests were performed to validate the statistical significance of our results.

For One Evader vs. Many Pursuers games, containment by calculating the area of the convex-hull formed by pursuers, checking if the evader was inside the hull using Delaunay triangulation was additionally measured. Further, conditional density plots were used to visualize this.

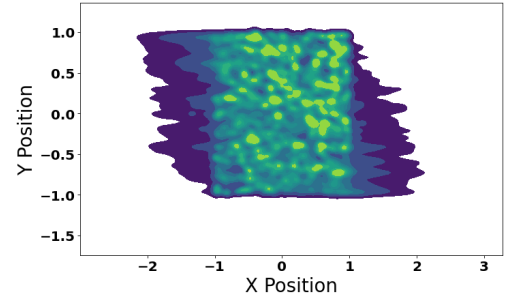
6.1.1 Many Evaders vs. Many Pursuers Results. Across all game types, hot starts reduced evader survival rates compared to random starts (see Appendix section G for full results). In just about every game configuration, the survival rate of evaders for strategies aided by hot starts dropped at a faster rate when compared to without them. Heatmaps revealed distinct movement patterns and activity areas for hot starts, mainly in the activeness of movement and area. Log-rank tests confirmed the statistical significance of our survival analysis, with, $p\text{-value} < .0001$ for all configurations.

6.1.2 One Evader vs. Many Pursuers Results. For One Evader vs. Many Pursuers games, the difference in survival rates between hot starts and random starts was less pronounced (see Appendix Section G for full results). Log-rank tests showed statistically significant differences, except in the 3 pursuers vs. 1 evader game. The heatmaps revealed similar movement patterns across starts, with differences mainly in movement frequency.

Several factors, including the diminishing returns of hot starts in high-pursuer-density scenarios, contribute to this outcome. The high density ensures quick evader interception, reducing the impact of initial configurations; essentially, since the ratio of pursuers to evaders is very high, an advantageous start has less of an impact on movement and the outcomes of the game.

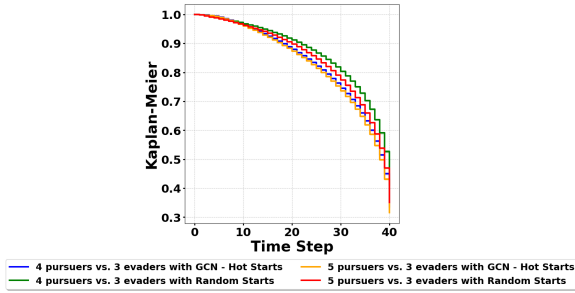


(a) 4 Pursuers vs. 2 Evaders GCN - Hot Starts heat map

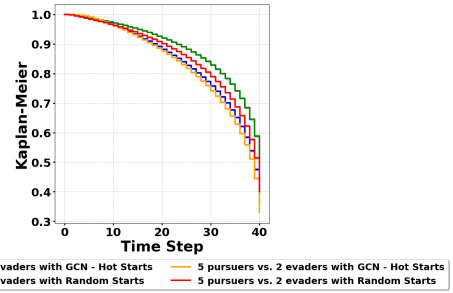


(b) 4 Pursuers vs. 2 Evaders Random Start heat map

Figure 4: Example Heat Maps for Many Pursuers vs. Many Evaders Games. The brighter the color, the more activity in that area of movement. See Appendix section G.4 for heatmaps for all Many Pursuers vs. Many Evaders Games.



(a) Survival Analysis of Game Simulations with 3 Evaders



(b) Survival Analysis of Game Simulations with 2 Evaders

Figure 5: Survival Analysis for Many Pursuers vs. Many Evaders Game Simulations. The x-axis is the time step and the y-axis is the value of the Kaplan-Meier Estimator.

6.2 Discussion

6.2.1 Many Evaders vs. Many Pursuers. Hot starts provided a clear advantage, as evader survival rates consistently dropped faster compared to random starts, as seen in figures 5a-b (see Table ?? for a direct example of 4 Pursuers vs. 2 Evaders games). The statistical significance of these results, supported by log-rank tests with p-values near zero, indicates that hot starts substantially influence game outcomes, favoring pursuers. This highlights the efficacy of our method in capturing and leveraging the spatio-temporal dynamics of agents, allowing pursuers to gain a strategic upper hand.

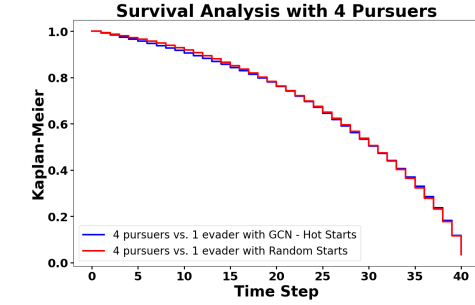
Moreover, heatmaps revealed that hot starts naturally led to strategic partitioning of the environment, such as dividing the space into distinct regions (e.g., diagonal lines, crescent shapes, or "S" curves, see figure 1 for an example). This partitioning was not pre-programmed but emerged as the GCN learned latent configurations that strategically positioned pursuers from the outset. These emergent patterns ensured that pursuers were never entirely out of position, even in scenarios where random evader placement might suggest no clear advantage. The strategic placement resulted in enhanced capture rates, demonstrating the power of the GCN in discovering and generating effective configurations that maximize pursuit efficiency over many games.

6.2.2 One Evader vs. Many Pursuers. The survival analysis and heatmaps for one evader versus many pursuers differed notably

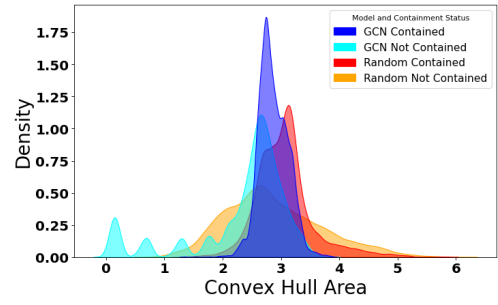
from the many vs. many games. Although the visual differences in survival rates between hot starts and random starts were minimal, log-rank tests still indicated statistically significant results (p-value < 0.05) in all but one game scenario (in 3 pursuers vs. 1 evader games there was no statistical significance). This suggests that, while hot starts have a statistically measurable impact on evader survival, the practical effect is less pronounced due to the overwhelming number of pursuers, which rapidly neutralizes the advantage. High pursuer density ensures that evaders are intercepted quickly, regardless of initial configuration, leading to similar containment outcomes in both hot and random starts.

6.2.3 Future Work and Limitations. This study serves as a proof of concept, establishing the foundational framework for our approach. Future work will concentrate on scaling the network, exploring additional game formats, expanding to larger environments, and applying our methodology to domains such as networked economics, robot swarms, and population initialization. Additionally, we are investigating the incorporation of Generative Adversarial Networks (GANs), Temporal Graph Networks, much more expressive GFS features, and human-in-the-loop systems similar to TacticAI.

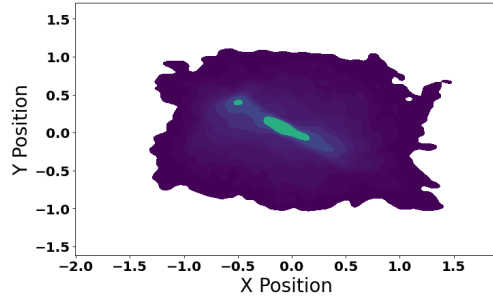
While our evaluation spanned a diverse range of game types, limitations remain primarily due to the simplified environments chosen for public accessibility. As a result, the generalizability of hot starts to more complex or varied game scenarios is still an



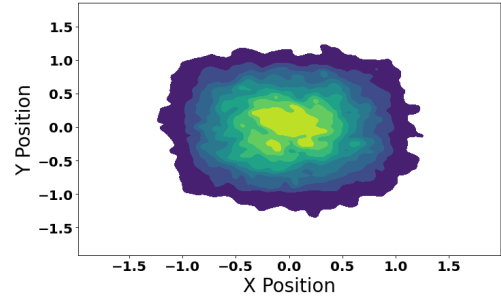
(a) 4 Pursuers vs. 1 Evader Survival Analysis



(b) 4 Pursuers vs. 1 Evader Conditional Density plot of containment



(c) 4 Pursuers vs. 1 Evader Hot Start heat map



(d) 4 Pursuers vs. 1 Evader Random Start heat map

Figure 6: Survival Analysis and movement analysis for 4 pursuers vs. 1 evader games. See Appendix section G.1 and G.2 for movement analysis and survival analysis tables for all 1 Evader vs. Many Pursuers games.

open question. Additionally, the current implementation of the Graph Feature Space relies on heuristics that, while effective for initial studies, lack the sophistication required for capturing all nuances of the underlying dynamics. Future work will focus on integrating the costs derived from the Hamilton-Jacobi-Isaacs (HJI) framework into the objective space, which we expect to yield a more robust performance metric. Furthermore, exploring the impact of hot starts on computing open-loop equilibria represents a promising theoretical direction, potentially offering deeper insights into the dynamics of pursuit-evasion games. d

7 CONCLUSION

This paper presents a novel framework that addresses the problem of strategically placing pursuers when evader locations are unknown, using "hot starts." Hot starts are generated by applying differential game theory algorithms, tracking pursuer movements, and modeling configurations through graphs. By solving a multi-objective optimization problem to identify Pareto-optimal graphs, then proceed to train a GCN on them, and use the GCN to generate hot starts. Empirical evaluations demonstrate that these hot starts offer advantages in terms of faster evader capture, improved containment, and optimized agent movement. Statistical analysis, including log-rank tests, confirms the significance of these results. While this work serves as a proof of concept, it provides a promising foundation for future research.

ACKNOWLEDGMENTS

This work was generously supported by the Air Force Research Laboratory and the Hap Arnold Scholars Fellowship. Special thanks to Dr. Alex Von Moll and Dr. Isaac Weintraub for their guidance as well.

REFERENCES

- [1] Jonathan Annas and Jing Xiao. 2009. Intelligent pursuit & evasion in an unknown environment. In *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 4899–4906.
- [2] Adonis Antoniadis, H Jin Kim, and Shankar Sastry. 2003. Pursuit-evasion strategies for teams of multiple agents with incomplete information. In *42nd IEEE International Conference on Decision and Control (IEEE Cat. No. 03CH37475)*, Vol. 1. IEEE, 756–761.
- [3] Efstathios Bakolas and Panagiotis Tsiotras. 2010. Optimal pursuit of moving targets using dynamic Voronoi diagrams. In *49th IEEE conference on decision and control (CDC)*. IEEE, 7431–7436.
- [4] Martino Bardi, Italo Capuzzo Dolcetta, et al. 1997. *Optimal control and viscosity solutions of Hamilton-Jacobi-Bellman equations*. Vol. 12. Springer.
- [5] Tamer Başar and Geert Jan Olsder. 1998. *Dynamic noncooperative game theory*. SIAM.
- [6] J. Blank and K. Deb. 2020. pymoo: Multi-Objective Optimization in Python. *IEEE Access* 8 (2020), 89497–89509.
- [7] Elizabeth Bondi, Fei Fang, Mark Hamilton, Debarun Kar, Donnabell Dmello, Jongmoo Choi, Robert Hannaford, Arvind Iyer, Lucas Joppa, Milind Tambe, et al. 2018. Spot poachers in action: Augmenting conservation drones with automatic detection in near real time. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [8] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation* 6, 2 (2002), 182–197.
- [9] Tomonari Furukawa, Frederic Bourgault, Benjamin Lavis, and Hugh F Durrant-Whyte. 2006. Recursive Bayesian search-and-tracking using coordinated UAVs

- for lost targets. In *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006*. IEEE, 2521–2526.
- [10] Eloy Garcia and Shaunak D. Bopardikar. 2021. Cooperative Containment of a High-speed Evader. In *2021 American Control Conference (ACC)*. 4698–4703. <https://doi.org/10.23919/ACC50511.2021.9483097>
- [11] Eloy Garcia, David W Casbeer, Alexander Von Moll, and Meir Pachter. 2020. Multiple pursuer multiple evader differential games. *IEEE Trans. Automat. Control* 66, 5 (2020), 2345–2350.
- [12] C Giovannangeli, M Heymann, E Rivlin, and V Kordic. 2010. Pursuit-Evasion Games in Presence of Obstacles in Unknown Environments: towards an optimal pursuit strategy. *Cutting edge robotics 2010* (2010), 47–80.
- [13] Manish Kumar Goel, Pardeep Khanna, and Jugal Kishore. 2010. Understanding survival analysis: Kaplan-Meier estimate. *International journal of Ayurveda research* 1, 4 (2010), 274.
- [14] Rufus Isaacs. 1969. Differential games: Their scope, nature, and future. *Journal of Optimization Theory and Applications* 3 (1969), 283–295.
- [15] Rufus Isaacs. 1999. *Differential games: a mathematical theory with applications to warfare and pursuit, control and optimization*. Courier Corporation.
- [16] Edward L Kaplan and Paul Meier. 1958. Nonparametric estimation from incomplete observations. *J. Amer. Statist. Assoc.* 53, 282 (1958), 457–481.
- [17] Borhan Kazimipour, Xiaodong Li, and A Kai Qin. 2014. A review of population initialization techniques for evolutionary algorithms. In *2014 IEEE congress on evolutionary computation (CEC)*. IEEE, 2585–2592.
- [18] Edward Keedwell, Mathieu Brevilliers, Lhassane Idoumghar, Julien Lepagnot, and Hojjat Rakhshani. 2018. A Novel Population Initialization Method Based on Support Vector Machine. In *2018 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 751–756. <https://doi.org/10.1109/SMC.2018.00136>
- [19] David W King, Jason M Bindewald, and Gilbert L Peterson. 2018. Informal team assignment in a pursuit-evasion game. In *The Thirty-First International Flairs Conference*.
- [20] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [21] Pengdeng Li, Shuxin Li, Xinrun Wang, Jakub Cerny, Youzhi Zhang, Stephen McAleer, Hau Chan, and Bo An. 2024. Grasper: A Generalist Pursuer for Pursuit-Evasion Problems. *arXiv preprint arXiv:2404.12626* (2024).
- [22] Shuxin Li, Xinrun Wang, Youzhi Zhang, Wanqi Xue, Jakub Černý, and Bo An. 2023. Solving large-scale pursuit-evasion games using pre-trained strategies. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 11586–11594.
- [23] Yang Li, Giovanni Carboni, Felipe Gonzalez, et al. 2019. Differential game theory for versatile physical human–robot interaction. *Nature Machine Intelligence* 1 (2019), 36–43. <https://doi.org/10.1038/s42256-018-0010-3>
- [24] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. 2017. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. *Neural Information Processing Systems (NIPS)* (2017).
- [25] Dejan Milutinović, David W. Casbeer, Alexander Von Moll, Meir Pachter, and Eloy Garcia. 2023. Rate of Loss Characterization That Resolves the Dilemma of the Wall Pursuit Game Solution. *IEEE Trans. Automat. Control* 68, 1 (2023), 242–256. <https://doi.org/10.1109/TAC.2021.3137786>
- [26] Ian M. Mitchell, Alexandre M. Bayen, and Claire J. Tomlin. 2005. A Time-Dependent Hamilton–Jacobi Formulation of Reachable Sets for Continuous Dynamic Games. *IEEE Trans. Automat. Control* 50, 7 (2005), 947–957. <https://doi.org/10.1109/TAC.2005.851439>
- [27] Igor Mordatch and Pieter Abbeel. 2017. Emergence of Grounded Compositional Language in Multi-Agent Populations. *arXiv preprint arXiv:1703.04908* (2017).
- [28] Stanley Osher and Ronald Fedkiw. 2003. *Level Set Methods and Dynamic Implicit Surfaces*. Applied Mathematical Sciences, Vol. 153. Springer. <https://doi.org/10.1007/b98879>
- [29] M V Ramana and Mangal Kothari. 2017. Pursuit-evasion games of high speed evader. *Journal of intelligent & robotic systems* 85 (2017), 293–306.
- [30] Jason T Rich, J Gail Neely, Randal C Paniello, Courtney CJ Voelker, Brian Nussenbaum, and Eric W Wang. 2010. A practical guide to understanding Kaplan-Meier curves. *Otolaryngology—Head and Neck Surgery* 143, 3 (2010), 331–336.
- [31] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2008. The graph neural network model. *IEEE transactions on neural networks* 20, 1 (2008), 61–80.
- [32] Milind Tambe. 2011. *Security and game theory: algorithms, deployed systems, lessons learned*. Cambridge university press.
- [33] Alexander Von Moll, David W Casbeer, Eloy Garcia, and Dejan Milutinović. 2018. Pursuit-evasion of an evader by multiple pursuers. In *2018 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 133–142.
- [34] Zhe Wang, Petar Veličković, Daniel Hennes, Nenad Tomašev, Laurel Prince, Michael Kaisers, Yoram Bachrach, Romuald Elie, Li Kevin Wenliang, Federico Piccinini, et al. 2024. TacticAI: an AI assistant for football tactics. *Nature communications* 15, 1 (2024), 1–13.
- [35] Isaac E Weintraub, Meir Pachter, and Eloy Garcia. 2020. An introduction to pursuit-evasion differential games. In *2020 American Control Conference (ACC)*. IEEE, 1049–1066.
- [36] Lily Xu, Shahrzad Gholami, Sara McCarthy, Bistra Dilkina, Andrew Plumptre, Milind Tambe, Rohit Singh, Mustapha Nsubuga, Joshua Mabonga, Margaret Driciru, et al. 2020. Stay ahead of poachers: Illegal wildlife poaching prediction and patrol planning under uncertainty with field test evaluations (short version). In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1898–1901.
- [37] Leiming Zhang, Amanda Prorok, and Subhrajit Bhattacharya. 2021. Pursuer assignment and control strategies in multi-agent pursuit-evasion under uncertainties. *Frontiers in Robotics and AI* 8 (2021), 691637.
- [38] Hongrui Zheng, Zhijun Zhuang, Stephanie Wu, Shuo Yang, and Rahul Mangharam. 2024. Bridging the Gap between Discrete Agent Strategies in Game Theory and Continuous Motion Planning in Dynamic Environments. *arXiv abs/2403.11334* (2024). <https://api.semanticscholar.org/CorpusID:268513329>
- [39] Eckart Zitzler, Dimo Brockhoff, and Lothar Thiele. 2007. The hypervolume indicator revisited: On the design of Pareto-compliant indicators via weighted integration. In *Evolutionary Multi-Criterion Optimization: 4th International Conference, EMO 2007, Matsushima, Japan, March 5-8, 2007. Proceedings 4*. Springer, 862–876.

Appendix

A Preliminaries

In this section, we provide some background on differential games and how they fit into the broader sphere of game theory and optimal control theory. The goal is to give readers with some knowledge of game theory or control theory an understanding of differential games using well-known and widely studied concepts. This section is adapted from *A Few Words About Differential Games* [5].

A.1 What is a Differential Game?

Games arise whenever two or more agents interact, each with their own goals. Typically, these goals are encoded by an objective cost function in static games (e.g., a simple utility function) or a cost functional in dynamic games. When agents operate within a system expressed through differential equations, it becomes a differential game. The field of differential game theory encompasses solution concepts, methodologies, applications, and specific instances of differential games, with these concepts often existing in different forms in other areas of control and game theory [2].

A.2 Equilibrium Concepts

A.2.1 Nash Equilibrium

John Nash, along with John von Neumann and others, introduced the concept of an equilibrium, broadly defined as a set of strategies where, holding all other agents' strategies constant, no agent can improve their outcome. This is famously known as the Nash equilibrium:

$$J(u^*, v^*) \leq J(u, v^*) \quad \forall u \in \mathbb{U} \quad (1)$$

where J is a cost function, u is the ego agent's strategy, and v^* is the equilibrium strategy. $\mathbb{U}$ represents the set of possible strategies for the ego agent.

A.2.2 Other Equilibria

Nash's equilibrium, originally described for static games, extends to differential games, albeit with increased computational complexity. The solution for

a differential game must be tailored to the specific information structure used in the problem formulation. For example, an open-loop Nash equilibrium can exist where players decide on specific control strategies *ex ante* and then "close their eyes" and let the game play out, without updating their strategies or reacting to changes. A more common and practical concept is the state-feedback Nash equilibrium, where player strategies can be a function of the current state. Interestingly, Isaacs [2] argued that it's not necessary to be in an equilibrium strategy at every moment. Other concepts, such as Stackelberg equilibrium, involve a *leader* who announces their strategy to *followers*, who then respond. The leader's strategy minimizes their own cost, given the followers' responses. These concepts generally apply to perfect information games, where all players have complete knowledge of the game state (though not necessarily of each other's control policies).

A.3 Zero-Sum Games

A zero-sum game is a type of constant-sum game where the sum of the outcomes of the players' strategies always equals zero. In these games, the Nash equilibrium is a saddle-point. For example, consider a cost functional J , where the u -agent seeks to minimize and the v -player seeks to maximize. The saddle-point equilibrium pair of strategies (u^*, v^*) satisfies:

$$J(u^*, v) \leq J(u^*, v^*) \leq J(u, v^*) \quad \forall u \in \mathbb{U}, \forall v \in \mathbb{V} \quad (2)$$

In this case, the strategy u^* is robust to any possible v that could be used, and vice versa. This is a stronger condition than in general-sum Nash equilibrium, where multiple players could defect and alter the outcome unfavorably.

A.4 Connecting to Control

In optimal control, there is typically a single agent with a single cost functional, meaning there are no other decision-makers or control signals in the system. The agent simply seeks to minimize or maximize their cost. One could think of optimal control as a specific type of differential game with one player, or conversely, view differential game theory as optimal control with more than one agent in the system.

A.5 Linear-Quadratic Games

When a game is described by a system of linear dynamics, such as:

$$\dot{\mathbf{x}} = A\mathbf{x} + B^u\mathbf{u} + B^v\mathbf{v} \quad (3)$$

with a quadratic cost functional:

$$J(\mathbf{x}, \mathbf{u}, \mathbf{v}) = \mathbf{x}_f^\top Q_f \mathbf{x}_f + \int_0^{t_f} (\mathbf{x}^\top Q_x \mathbf{x} + \mathbf{u}^\top R^u \mathbf{u} - \mathbf{v}^\top R^v \mathbf{v}) dt \quad (4)$$

Here, Q_f , Q_x , R^u , and R^v are positive semi-definite cost matrices. This is known as an LQ game. LQ games satisfy Isaacs' key condition for two-player differential games:

$$\min_{\mathbf{u}} \max_{\mathbf{v}} J(\mathbf{x}, \mathbf{u}, \mathbf{v}) = \max_{\mathbf{v}} \min_{\mathbf{u}} J(\mathbf{x}, \mathbf{u}, \mathbf{v}) \quad (5)$$

This holds because the two agents' controls are separable in the cost functional. This structure is well-established and often allows for analytical solutions, frequently connecting to the Riccati equations in LQ control problems. Researchers often use these games as initial base cases or testing grounds for new ideas.

A.6 Solutions to Differential Games

In differential games, we define the solution as a triple $(V(\mathbf{x}), \mathbf{u}^*, \mathbf{v}^*)$, where $V(\mathbf{x})$ is the value function, and $\mathbf{u}^*$ and $\mathbf{v}^*$ are the control strategies for the agents. The value function represents the equilibrium value of the cost functional when the game starts from some initial conditions. For V to be valid, it must be continuously differentiable (a condition we will touch on later) and satisfy the well-known Hamilton-Jacobi-Isaacs (HJI) partial differential equation:

$$\min_{\mathbf{u}} \max_{\mathbf{v}} \left[l(\mathbf{x}, \mathbf{u}, \mathbf{v}, t) + \frac{\partial V}{\partial t} + \nabla_{\mathbf{x}} V \cdot \mathbf{f}(\mathbf{x}, \mathbf{u}, \mathbf{v}, t) \right] = 0 \quad (6)$$

where l is the running cost and $\mathbf{f} = \dot{\mathbf{x}}$. Solving the HJI equation directly is typically infeasible, so alternative solution concepts are often employed.

A.6.1 Isaacs' Verification Theorem

In Isaacs' key works, he mentions that it suffices to check whether a candidate value function satisfies the HJI. If it does, then it is a solution to the differential game. One approach often taken by scientists and mathematicians is to guess a strategy for $\mathbf{u}$ and $\mathbf{v}$ (akin to selecting heading angles and velocities as in our Hot Starts paper), substitute these into V , and then substitute V into the HJI.

A.7 Method of Characteristics

The Method of Characteristics (MoC) is akin to the indirect method of optimal control, relying on the backward integration of the equilibrium state and costate dynamics from the terminal state set. The Hamiltonian of the system is defined as:

$$\mathbb{H} = \boldsymbol{\lambda} \cdot \mathbf{f} + l \quad (7)$$

where $\boldsymbol{\lambda}$ is the vector of costates, and l is the running cost. From this, we can derive the equilibrium control as a function of costate variables with the necessary conditions for equilibrium:

$$\mathbf{u}^* = \min_{\mathbf{u}} \mathbb{H}, \quad \mathbf{v}^* = \max_{\mathbf{v}} \mathbb{H} \quad (8)$$

Ultimately, the terminal costate values can be expressed as a function of the terminal position, allowing for the backward integration of the system's state and costate to capture an equilibrium trajectory (analogous to backward induction in solving a Stackelberg game). This process is repeated over the terminal manifold, essentially filling the space with equilibrium trajectories.

A.8 Singularities in the Environment

Although our work did not address this (as we had no obstacles in our environment), we felt it was important to include a section on singularities. Earlier, we mentioned a caveat regarding the regularity requirement of value functions. It is possible that there may be one or more discontinuous surfaces in the environment, or they may not be continuously differentiable. Their presence poses a problem for MoC. If, for a particular terminal state, one were to backward integrate, any trajectory that continues beyond the singularity is incorrect (no longer part of the equilibrium trajectory). This is a significant area of research.

B Control Algorithms for Agent Movement

B.1 Notation

- P_i : Pursuer i
- E_j : Evader j in many vs. many games
- E : Evader in one vs. many games
- (x_E, y_E) : Position of the evader
- $\{(x_{P_i}, y_{P_i})\}$: Positions of the pursuers
- $\{(x_{E_j}, y_{E_j})\}$: Positions of the evaders in many vs. many games
- v_E : Velocity of the evader (constant .5)
- v_P : Velocity of the pursuers (constant 1)
- k : current time step
- Δt : Change in Time step
- ρ : Capture radius
- γ : Speed ratio ($\gamma = \frac{v_E}{v_P}$)
- ϕ_i, ϕ_j : Angles formed by the line connecting the evader to P_i and P_j
- λ_i, λ_j : Line-of-sight angles from pursuers to the evader
- θ_{ij} : Weakest link angle between two pursuers P_i and P_j
- Ψ_{Es}, Ψ_{Ec} : Functions of distances and angles to the pursuers
- ψ_E : Optimal heading angle for the evader
- d : Distance between pursuer P_i and evader
- θ_{optimal} : Optimal heading angle for pursuer P_i
- θ_p : Optimal heading angle for the pursuer
- λ_x, λ_y : Co-states corresponding to the x and y coordinates
- H_p : Hamiltonian of the pursuer
- H_E : Hamiltonian of the evader
- V : Value function

B.2 One Evader vs. Many Pursuers

B.2.1 Evader Heuristic Control Algorithm

High-Level Description: The evader identifies the weakest link between pursuers by computing the overlapping angle between each pair of pursuers. The pair with the smallest overlapping angle is deemed the weakest link. The evader then computes the optimal heading angle to exploit this weakest link and escape.

Algorithm 1 Evader Control Algorithm for One Evader vs. Many Pursuers Game (Discrete Time)

- 1: **Input:** $(x_E[k], y_E[k]), \{(x_{P_i}[k], y_{P_i}[k])\}, v_E, v_P, \Delta t, \rho, \gamma$
 - 2: **Output:** $(x_E[k+1], y_E[k+1])$
 - 3: Initialize $\text{min_theta} \leftarrow \infty$
 - 4: Initialize $\text{weakest_pair} \leftarrow \text{None}$
 - 5: **for** each pair of pursuers (P_i, P_j) **do**
 - 6: Calculate $d_i[k] = \sqrt{(x_{P_i}[k] - x_E[k])^2 + (y_{P_i}[k] - y_E[k])^2}$
 - 7: Calculate $d_j[k] = \sqrt{(x_{P_j}[k] - x_E[k])^2 + (y_{P_j}[k] - y_E[k])^2}$
 - 8: Calculate $\phi_i[k] = \begin{cases} \arccos\left(\frac{1-d_i[k]^2}{2d_i[k]^2}\right) & \text{if } d_i[k]^2 > 1 \\ 0 & \text{otherwise} \end{cases}$
 - 9: Calculate $\phi_j[k] = \begin{cases} \arccos\left(\frac{1-d_j[k]^2}{2d_j[k]^2}\right) & \text{if } d_j[k]^2 > 1 \\ 0 & \text{otherwise} \end{cases}$
 - 10: Calculate $\lambda_i[k] = \arctan 2(y_{P_i}[k] - y_E[k], x_{P_i}[k] - x_E[k])$
 - 11: Calculate $\lambda_j[k] = \arctan 2(y_{P_j}[k] - y_E[k], x_{P_j}[k] - x_E[k])$
 - 12: Calculate $\theta_{ij}[k] = \phi_i[k] + \phi_j[k] - (\lambda_i[k] - \lambda_j[k])$
 - 13: **if** $\theta_{ij}[k] < \text{min_theta}$ **then**
 - 14: $\text{min_theta} \leftarrow \theta_{ij}[k]$
 - 15: $\text{weakest_pair} \leftarrow (P_i, P_j)$
 - 16: **end if**
 - 17: **end for**
 - 18: **if** $\text{weakest_pair} \neq \text{None}$ **then**
 - 19: Let $(P_i, P_j) \leftarrow \text{weakest_pair}$
 - 20: Calculate $\Psi_{Es}[k] = \frac{1}{d_j[k]} \left(\cos(\lambda_j[k]) - \frac{\sin(\lambda_j[k])}{\sqrt{\max(d_j[k]^2 - 2, 10^{-6})}} \right) - \frac{1}{d_i[k]} \left(\cos(\lambda_i[k]) + \frac{\sin(\lambda_i[k])}{\sqrt{\max(d_i[k]^2 - 2, 10^{-6})}} \right)$
 - 21: Calculate $\Psi_{Ec}[k] = \frac{1}{d_j[k]} \left(\sin(\lambda_i[k]) - \frac{\cos(\lambda_i[k])}{\sqrt{\max(d_i[k]^2 - 2, 10^{-6})}} \right) - \frac{1}{d_i[k]} \left(\sin(\lambda_j[k]) + \frac{\cos(\lambda_j[k])}{\sqrt{\max(d_j[k]^2 - 2, 10^{-6})}} \right)$
 - 22: Calculate $\psi_E[k] = \arctan 2(\Psi_{Es}[k], \Psi_{Ec}[k])$
 - 23: **end if**
 - 24: Update the evader's position:
 - 25: $x_E[k+1] = x_E[k] + v_E \cos(\psi_E[k])\Delta t$
 - 26: $y_E[k+1] = y_E[k] + v_E \sin(\psi_E[k])\Delta t$
-

B.2.2 Pursuers' Control Algorithm

High-Level Description: Pursuers are divided into active and non-active based on their proximity to the evader. Active pursuers compute their optimal heading angle towards the evader while considering the weakest link between pursuers. Non-active pursuers compute the heading angle to a support position to contain the evader.

Algorithm 2 Pursuer Control Algorithm for One Evader vs. Many Pursuers Game (Discrete Time)

```

1: Input:  $(x_E[k], y_E[k]), \{(x_{P_i}[k], y_{P_i}[k])\}, v_P, \Delta t, \rho, \gamma$ 
2: Output:  $\{(x_{P_i}[k+1], y_{P_i}[k+1])\}$ 
3: for each pursuer  $P_i$  do
4:   Calculate  $d[k] = \sqrt{(x_E[k] - x_{P_i}[k])^2 + (y_E[k] - y_{P_i}[k])^2}$ 
5:   if  $d[k] \leq \rho$  then
6:      $x_{P_i}[k+1] = x_{P_i}[k]$ 
7:      $y_{P_i}[k+1] = y_{P_i}[k]$ 
8:   else
9:      $x_{P_i}^{[k]} = x_{P_i}[k] + \frac{\gamma(x_E[k] - x_{P_i}[k]) + \rho}{1 - \gamma^2}$ 
10:     $y_{P_i}^{[k]} = y_{P_i}[k] + \frac{\gamma(y_E[k] - y_{P_i}[k]) + \rho}{1 - \gamma^2}$ 
11:   end if
12:   Calculate  $\theta_{\text{optimal}}[k] = \arctan 2 \left( y_{P_i}^{[k]} - y_{P_i}[k], x_{P_i}^{[k]} - x_{P_i}[k] \right)$ 
13:   Update the pursuer's position:
14:    $x_{P_i}[k+1] = x_{P_i}[k] + v_P \cos(\theta_{\text{optimal}}[k])\Delta t$ 
15:    $y_{P_i}[k+1] = y_{P_i}[k] + v_P \sin(\theta_{\text{optimal}}[k])\Delta t$ 
16: end for

```

B.3 Many Evaders vs. Many Pursuers

For many-vs-many scenarios ($\gamma = \frac{1}{2} < 1$), where pursuers are faster than evaders, we employ a hybrid approach combining geometry-based controls with PDE solutions. Specifically, we use a first-order upwind finite difference scheme with directional bias: backward differences are selected for negative gradients and forward differences for positive gradients. This scheme provides numerical stability while respecting the information flow direction in the HJI equation, consistent with methods described in [4, 6]. The Hamiltonian computation explicitly accounts for the minimizing pursuer and maximizing evader roles, ensuring appropriate upwind direction selection based on agent type.

B.3.1 Geometric Measures and Hybrid Control

Our hybrid approach uses a weighted parameter $\alpha = 0.5$ to combine geometric distance metrics with PDE value function information in the assignment cost matrix for the Hungarian algorithm. The geometric measures specifically include:

1. Euclidean distances between pursuers and evaders, used in the assignment

cost matrix:

$$d_{ij} = \|\mathbf{p}_i - \mathbf{e}_j\|_2 \quad (9)$$

2. Normalized direction vectors for determining movement trajectories:

$$\hat{\mathbf{d}}_{ij} = \frac{\mathbf{e}_j - \mathbf{p}_i}{\|\mathbf{e}_j - \mathbf{p}_i\|_2} \quad (10)$$

3. Relative bearing angles for optimal interception paths:

$$\theta_{ij} = \arctan 2(\hat{\mathbf{d}}_{ij}[1], \hat{\mathbf{d}}_{ij}[0]) \quad (11)$$

4. Proximity-based avoidance vectors that allow evaders to compute escape directions:

$$\mathbf{a}_j = \sum_{i=1}^{N_p} \frac{\mathbf{e}_j - \mathbf{p}_i}{\|\mathbf{e}_j - \mathbf{p}_i\|_2^2} \cdot \mathbf{1}_{\|\mathbf{e}_j - \mathbf{p}_i\| < r_{avoid}} \quad (12)$$

Pursuers are allocated to evaders by minimizing this hybrid cost using the Hungarian algorithm, and control actions are computed by combining gradient information from the PDE with direct geometric vectors:

$$\mathbf{u}_{final} = \alpha \cdot \mathbf{u}_{geometric} + (1 - \alpha) \cdot \mathbf{u}_{gradient} \quad (13)$$

where $\mathbf{u}_{geometric}$ is derived from the instantaneous geometric relationship between agents, and $\mathbf{u}_{gradient}$ comes from the spatial derivatives of the value function.

B.3.2 Upwind Scheme for HJI Equation

We compute the upwind derivatives of the value function as follows:

$$V_x^{\text{forward}} = \frac{V(i+1, j) - V(i, j)}{\Delta x} \quad (14)$$

$$V_x^{\text{backward}} = \frac{V(i, j) - V(i-1, j)}{\Delta x} \quad (15)$$

$$V_y^{\text{forward}} = \frac{V(i, j+1) - V(i, j)}{\Delta y} \quad (16)$$

$$V_y^{\text{backward}} = \frac{V(i, j) - V(i, j-1)}{\Delta y} \quad (17)$$

Then, for pursuers (minimizing player) and evaders (maximizing player), we select the appropriate derivatives:

$$V_x^{\text{pursuer}} = \begin{cases} V_x^{\text{backward}} & \text{if } V_x^{\text{forward}} \geq 0 \\ V_x^{\text{forward}} & \text{otherwise} \end{cases} \quad (18)$$

$$V_x^{\text{evader}} = \begin{cases} V_x^{\text{forward}} & \text{if } V_x^{\text{backward}} \geq 0 \\ V_x^{\text{backward}} & \text{otherwise} \end{cases} \quad (19)$$

This directional bias ensures numerical stability while correctly modeling the min-max nature of the game.

B.3.3 System Dynamics and Value Function Evolution

The system dynamics for the Many Evaders vs. Many Pursuers game in discrete time are defined as follows. Let there be N_p pursuers and N_e evaders. The state vector $\mathbf{x}[k]$ at time step k comprises the positions of all pursuers and evaders:

$$\mathbf{x}[k] = \begin{bmatrix} x_{P_1}[k] \\ y_{P_1}[k] \\ \vdots \\ x_{P_{N_p}}[k] \\ y_{P_{N_p}}[k] \\ x_{E_1}[k] \\ y_{E_1}[k] \\ \vdots \\ x_{E_{N_e}}[k] \\ y_{E_{N_e}}[k] \end{bmatrix}$$

The discrete-time system dynamics $\mathbf{x}[k+1]$ are given by:

$$\mathbf{x}[k+1] = \mathbf{x}[k] + \Delta t \cdot \mathbf{f}(\mathbf{x}[k], \mathbf{u}_p[k], \mathbf{u}_E[k])$$

where the control inputs $\mathbf{u}_p[k]$ for pursuers and $\mathbf{u}_E[k]$ for evaders are derived from our hybrid approach.

The discrete-time HJI equation is formulated as:

$$V[k+1] = V[k] - \Delta t \cdot H(\mathbf{x}[k], \nabla V[k])$$

where H is the Hamiltonian:

$$H(\mathbf{x}, \nabla V) = \min_{\mathbf{u}_p} \max_{\mathbf{u}_E} \nabla V \cdot \mathbf{f}(\mathbf{x}, \mathbf{u}_p, \mathbf{u}_E)$$

Evaders employ escape strategies based on gradient ascent of the value function combined with geometric avoidance of nearby pursuers, computing avoidance vectors that are inversely proportional to the squared distance from approaching pursuers when they enter a predefined avoidance radius.

C Environment

The environment in use was the PettingZoo Simple Tag MPE Environment. This is a predator prey environment, it's a discrete time step environment and is a continuous space environment. We used the environment without obstacles, and augmented the environment to our need; we also ignored rewards related to collisions, as those were not needed for our situation. With the use of open

source tools, we aim to make our code available to the public once it’s fully gone through the release process.

D Training Results

In this section, we include our training results for a batch size of 1024, learning rate of $1e^{-5}$, Adam optimizer, weight decay of $1e^{-2}$, and we trained for 150 epochs. We trained our network on an L4 GPU, which took about 2 hours. Over training, we measured GD, GD+, IGD, and IGD+ [1] to give us an idea of how the generated graphs relate to our Pareto values from our GCS. Generational Difference (GD) and Generational Difference Plus (GD+), can be expressed as:

$$GD = \left(\frac{1}{|P|} \sum_{x \in P} (d(x, P^*))^p \right)^{1/p} \quad (20)$$

$$GD^+ = \left(\frac{1}{|P|} \sum_{x \in P} (d^+(x, P^*))^p \right)^{1/p} \quad (21)$$

where P is the set of approximate Pareto-optimal solutions, P^* is the set of true Pareto-optimal solutions, $d(x, P^*)$ is the minimum Euclidean distance from

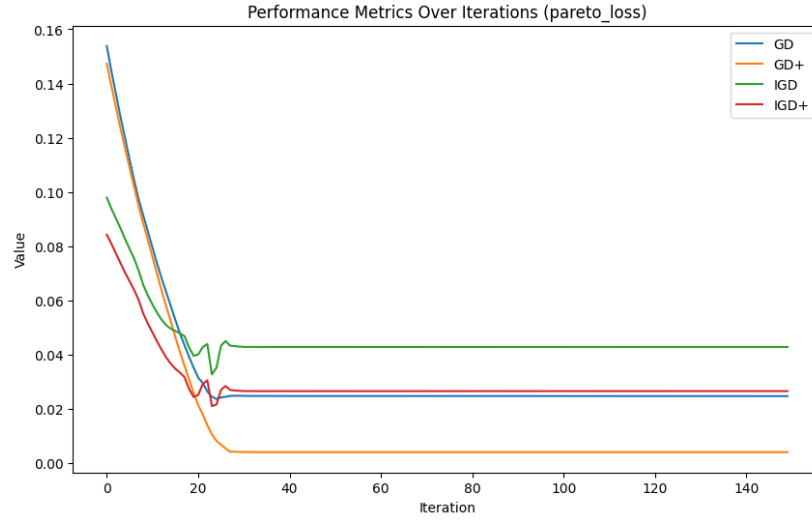


Figure 1: Generational Difference (GD), Generational Difference Plus (GD+), Inverse Generational Difference (IGD), and Inverse Generational Difference Plus (IGD+).

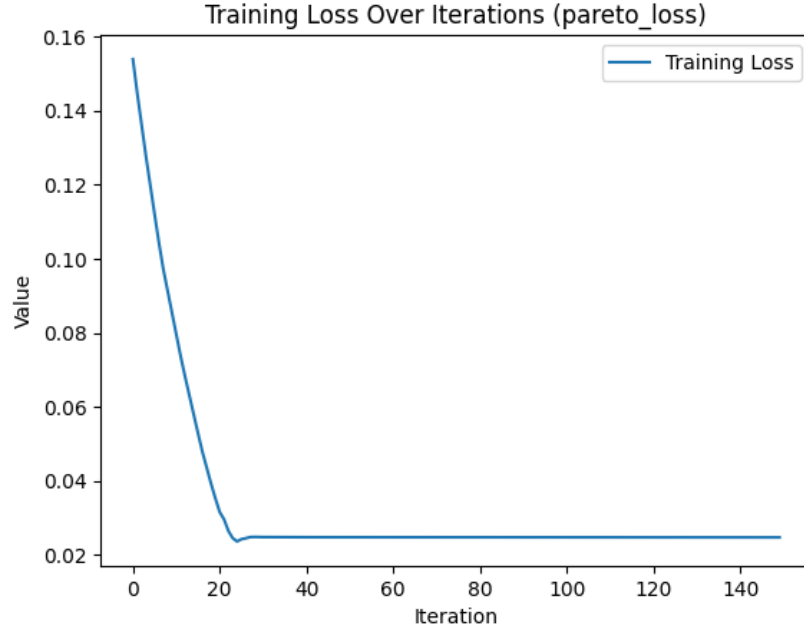


Figure 2: Pareto Loss during GCN training.

Figure 3: Training metrics for GCN: GD, GD+, IGD, IGD+, and Pareto Loss.

solution x to the true Pareto front P^* , $d^+(x, P^*)$ is the perpendicular distance from x to P^* , and p is usually set to 2.

Inverse Generational Difference (IGD) and Inverse Generational Difference Plus (IGD+) are expressed as:

$$IGD = \left(\frac{1}{|P^*|} \sum_{x^* \in P^*} (d(x^*, P))^p \right)^{1/p} \quad (22)$$

$$IGD^+ = \left(\frac{1}{|P^*|} \sum_{x^* \in P^*} (d^+(x^*, P))^p \right)^{1/p} \quad (23)$$

where $d(x^*, P)$ is the minimum Euclidean distance from the true Pareto-optimal solution x^* to the approximate Pareto front P .

Finally, Hypervolume (HV) is expressed as:

$$HV = \text{Lebesgue Measure} \left(\bigcup_{x \in P} [f_1(x), r_1] \times [f_2(x), r_2] \times \cdots \times [f_m(x), r_m] \right) \quad (24)$$

where $r = (r_1, r_2, \dots, r_m)$ is a reference point dominated by all Pareto-optimal solutions, and $f_i(x)$ is the i -th objective function value of solution x .

E Mathematical Examples of U_{heading} and U_{capture}

To illustrate the correctness and behavior of U_{capture} and U_{heading} , consider the following examples:

Example 1: U_{capture}

- Scenario 1: $d_i = r$

$$\begin{aligned}
U_{\text{capture}} &= 2 \left(1 - \frac{1}{1 + e^{\frac{-r+r}{r}}} \right) \\
&= 2 \left(1 - \frac{1}{1 + e^0} \right) \\
&= 2 \left(1 - \frac{1}{2} \right) \\
&= 2 \times 0.5 \\
&= 1
\end{aligned}$$

Interpretation: When the pursuer is exactly at the capture radius distance from the evader, the capture potential U_{capture} is 1.

- Scenario 2: $d_i < r$ (e.g., $d_i = 0$)

$$\begin{aligned}
U_{\text{capture}} &= 2 \left(1 - \frac{1}{1 + e^{\frac{-0+r}{r}}} \right) \\
&= 2 \left(1 - \frac{1}{1 + e^1} \right) \\
&= 2 \left(1 - \frac{1}{1 + 2.71828} \right) \\
&= 2 \left(1 - \frac{1}{3.71828} \right) \\
&= 2 \times 0.73206 \\
&= 1.46412
\end{aligned}$$

Interpretation: When the pursuer is at the same position as the evader ($d_i = 0$), the capture potential U_{capture} exceeds 1, indicating a higher likelihood of capture.

- Scenario 3: $d_i > r$ (e.g., $d_i = 2r$)

$$\begin{aligned}
U_{\text{capture}} &= 2 \left(1 - \frac{1}{1 + e^{\frac{-2r+r}{r}}} \right) \\
&= 2 \left(1 - \frac{1}{1 + e^{-1}} \right) \\
&= 2 \left(1 - \frac{1}{1 + 0.36788} \right) \\
&= 2 \left(1 - \frac{1}{1.36788} \right) \\
&= 2 \times 0.26768 \\
&= 0.53536
\end{aligned}$$

Interpretation: When the pursuer is twice the capture radius distance away from the evader, the capture potential U_{capture} decreases to approximately 0.535, indicating a lower likelihood of capture.

Example 2: U_{heading}

- Scenario 1: Perfect Alignment ($\theta = 0^\circ$)

$$\mathbf{v}_p = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (\text{Velocity vector pointing along the positive x-axis})$$

$$\mathbf{e}_1 - \mathbf{p} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (\text{Evader is directly ahead})$$

$$\theta = \arccos \left(\frac{\mathbf{v}_p \cdot (\mathbf{e}_1 - \mathbf{p})}{\|\mathbf{v}_p\| \|\mathbf{e}_1 - \mathbf{p}\| + \epsilon} \right)$$

$$= \arccos \left(\frac{1 \times 1 + 0 \times 0}{1 \times 1 + 10^{-9}} \right)$$

$$\approx \arccos(1)$$

$$\approx 0^\circ$$

$$U_{\text{heading}} \approx \frac{1}{1} \times 0^\circ$$

$$\approx 0^\circ$$

Interpretation: When the pursuer's heading is perfectly aligned with the direction to the evader, U_{heading} is 0, indicating optimal alignment for direct interception.

- Scenario 2: Partial Alignment ($\theta = 60^\circ$)

$$\mathbf{v}_p = \begin{bmatrix} \cos(60^\circ) \\ \sin(60^\circ) \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.8660 \end{bmatrix}$$

$$\mathbf{e}_1 - \mathbf{p} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\theta = \arccos \left(\frac{0.5 \times 1 + 0.8660 \times 0}{\sqrt{0.5^2 + 0.8660^2} \times \sqrt{1^2 + 0^2} + \epsilon} \right)$$

$$= \arccos \left(\frac{0.5}{1 + 10^{-9}} \right)$$

$$\approx \arccos(0.5)$$

$$\approx 60^\circ$$

$$U_{\text{heading}} \approx \frac{1}{1} \times 60^\circ$$

$$\approx 60^\circ$$

Interpretation: When the pursuer's heading forms a 60° angle with the direction to the evader, U_{heading} is 60° , indicating a moderate alignment.

- Scenario 3: Orthogonal Alignment ($\theta = 90^\circ$)

$$\begin{aligned}
\mathbf{v}_p &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} && \text{(Velocity vector pointing along the positive y-axis)} \\
\mathbf{e}_1 - \mathbf{p} &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\
\theta &= \arccos\left(\frac{0 \times 1 + 1 \times 0}{\sqrt{0^2 + 1^2} \times \sqrt{1^2 + 0^2} + \epsilon}\right) \\
&= \arccos(0) \\
&= 90^\circ \\
U_{\text{heading}} &= \frac{1}{1} \times 90^\circ \\
&= 90^\circ
\end{aligned}$$

Interpretation: When the pursuer's heading is orthogonal to the direction to the evader, U_{heading} is 90° , indicating no alignment.

- Scenario 4: Opposite Alignment ($\theta = 180^\circ$)

$$\begin{aligned}
\mathbf{v}_p &= \begin{bmatrix} -1 \\ 0 \end{bmatrix} && \text{(Velocity vector pointing along the negative x-axis)} \\
\mathbf{e}_1 - \mathbf{p} &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\
\theta &= \arccos\left(\frac{-1 \times 1 + 0 \times 0}{\sqrt{(-1)^2 + 0^2} \times \sqrt{1^2 + 0^2} + \epsilon}\right) \\
&\approx \arccos(-1) \\
&\approx 180^\circ \\
U_{\text{heading}} &\approx \frac{1}{1} \times 180^\circ \\
&\approx 180^\circ
\end{aligned}$$

Interpretation: When the pursuer's heading is directly opposite to the direction to the evader, U_{heading} is 180° , indicating maximum misalignment.

F Kaplan-Meier Estimator

In our work, the Kaplan-Meier estimator is used to estimate the survival probability of an evader over the course of the game. The Kaplan-Meier estimator is commonly used to estimate the survival probability of cancer patients or to determine how long a piece of machinery will last [7, 3]. The formula is expressed as:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \quad (25)$$

where:

- t_i are the ordered times at which events (evader captures) occur.
- d_i is the number of evaders captured at time t_i .
- n_i is the number of evaders at risk just before time t_i (essentially the evader still alive).

F.1 Application in Our Study

The Kaplan-Meier estimator provides a step-wise estimate of the survival probability $\hat{S}(t)$, accounting for censored data—evaders that are not captured within the observation period. By applying this estimator to our simulation data, we were able to:

- **Estimate Evader Survival Probabilities:** Calculate the probability that an evader remains uncaptured up to each time point t .
- **Compare Pursuer Configurations:** Analyze and compare the effectiveness of different pursuer configurations (e.g., hot starts vs. random starts) based on the estimated survival probabilities.
- **Assess Statistical Significance:** Utilize the Kaplan-Meier survival curves in conjunction with log-rank tests to determine the statistical significance of differences observed between strategies.

This approach allowed us to rigorously evaluate the performance improvements introduced by our pursuer configuration strategies, ensuring that the observed benefits are both meaningful and statistically validated.

G Additional Plots and Tables

In this section, we include further plots and tables on movement in simulations. 4 Pursuer vs. 2 Evader games results, are given in the paper, but here we include more to further our point.

G.1 Many vs. Many Supplementary Tables

We see across the board the impact of hot starts on survival probability as in each case, by simply using a hot start we are able to reduce the survival probability of evaders across each game format tested.

Table 1: Survival Probabilities for 5 Pursuers vs. 2 Evaders

5 Pursuers vs. 2 Evaders with GCN - Hot Starts 5 Pursuers vs. 2 Evaders with Random Starts

Time	Survival Probability	Time	Survival Probability
0	1.000000	0	1.000000
1	0.999222	1	0.998020
2	0.998300	2	0.994668
3	0.996939	3	0.991215
4	0.994560	4	0.987615
5	0.990941	5	0.983908
6	0.986354	6	0.980029
7	0.980970	7	0.975901
8	0.974989	8	0.971578
9	0.968652	9	0.967084
10	0.961855	10	0.962386
11	0.954719	11	0.957494
12	0.947278	12	0.952343
13	0.939514	13	0.946922
14	0.931382	14	0.941247
15	0.922949	15	0.935277
16	0.914204	16	0.929030
17	0.905176	17	0.922492
18	0.895805	18	0.915537
19	0.886094	19	0.908179
20	0.875959	20	0.900417
21	0.865351	21	0.892182
22	0.854188	22	0.883387
23	0.842500	23	0.874053
24	0.830273	24	0.864181
25	0.817456	25	0.853671
26	0.803995	26	0.842459
27	0.789788	27	0.830490
28	0.774721	28	0.817720
29	0.758697	29	0.804057
30	0.741516	30	0.789255
31	0.723015	31	0.773004
32	0.702835	32	0.755122
33	0.680740	33	0.735234
34	0.656146	34	0.712862
35	0.628434	35	0.687413
36	0.596409	36	0.657744
37	0.558314	37	0.622012
38	0.510671	38	0.576916
39	0.445101	39	0.513946
40	0.330532	40	0.400981

Table 2: Survival Probabilities for 4 Pursuers vs. 3 Evaders

4 Pursuers vs. 3 Evaders with GCN -
Hot Starts

Time	Survival Probability
0	1.000000
1	0.999092
2	0.998024
3	0.996518
4	0.994117
5	0.990603
6	0.986178
7	0.980965
8	0.975210
9	0.969067
10	0.962570
11	0.955685
12	0.948512
13	0.941056
14	0.933264
15	0.925127
16	0.916647
17	0.907805
18	0.898566
19	0.888968
20	0.878970
21	0.868551
22	0.857614
23	0.846139
24	0.834086
25	0.821397
26	0.807958
27	0.793756
28	0.778695
29	0.762651
30	0.745433
31	0.726878
32	0.706741
33	0.684626
34	0.660110
35	0.632504
36	0.600679
37	0.562786
38	0.515387
39	0.450105
40	0.335590

4 Pursuers vs. 3 Evaders with Ran-
dom Starts

Time	Survival Probability
0	1.000000
1	0.998310
2	0.995495
3	0.992602
4	0.989600
5	0.986452
6	0.983164
7	0.979680
8	0.976016
9	0.972165
10	0.968093
11	0.963801
12	0.959254
13	0.954455
14	0.949379
15	0.943999
16	0.938291
17	0.932209
18	0.925741
19	0.918875
20	0.911560
21	0.903789
22	0.895473
23	0.886663
24	0.877266
25	0.867232
26	0.856392
27	0.844800
28	0.832285
29	0.818680
30	0.803850
31	0.787633
32	0.769758
33	0.749969
34	0.727702
35	0.702192
36	0.672407
37	0.636503
38	0.590763
39	0.526715
40	0.411715

Table 3: Survival Probabilities for 5 Pursuers vs. 3 Evaders

5 Pursuers vs. 3 Evaders with GCN - Hot Starts 5 Pursuers vs. 3 Evaders with Random Starts

Time	Survival Probability	Time	Survival Probability
0	1.000000	0	1.000000
1	0.998993	1	0.997978
2	0.997815	2	0.994643
3	0.996199	3	0.991232
4	0.993568	4	0.987706
5	0.989759	5	0.983977
6	0.984922	6	0.980093
7	0.979366	7	0.975994
8	0.973246	8	0.971676
9	0.966679	9	0.967206
10	0.959752	10	0.962549
11	0.952511	11	0.957687
12	0.944972	12	0.952562
13	0.937122	13	0.947123
14	0.928993	14	0.941317
15	0.920544	15	0.935106
16	0.911727	16	0.928461
17	0.902554	17	0.921335
18	0.893002	18	0.913756
19	0.883076	19	0.905696
20	0.872743	20	0.897185
21	0.861976	21	0.888120
22	0.850758	22	0.878488
23	0.838999	23	0.868210
24	0.826674	24	0.857298
25	0.813681	25	0.845724
26	0.799921	26	0.833380
27	0.785332	27	0.820070
28	0.769855	28	0.805687
29	0.753342	29	0.790146
30	0.735631	30	0.773343
31	0.716578	31	0.755066
32	0.695787	32	0.734925
33	0.672959	33	0.712675
34	0.647618	34	0.687779
35	0.619072	35	0.659481
36	0.586154	36	0.626621
37	0.547038	37	0.587363
38	0.498303	38	0.538064
39	0.431647	39	0.469927
40	0.316023	40	0.350472

Table 4: Survival Probabilities for 4 Pursuers vs. 4 Evaders

4 Pursuers vs. 4 Evaders with GCN - Hot Starts 4 Pursuers vs. 4 Evaders with Random Starts

Time	Survival Probability	Time	Survival Probability
0	1.000000	0	1.000000
1	0.999123	1	0.998219
2	0.998092	2	0.995346
3	0.996676	3	0.992389
4	0.994383	4	0.989340
5	0.991085	5	0.986164
6	0.986928	6	0.982834
7	0.982059	7	0.979360
8	0.976650	8	0.975713
9	0.970822	9	0.971886
10	0.964563	10	0.967864
11	0.958040	11	0.963565
12	0.951187	12	0.958984
13	0.944028	13	0.954065
14	0.936554	14	0.948824
15	0.928742	15	0.943268
16	0.920618	16	0.937344
17	0.912157	17	0.931068
18	0.903338	18	0.924379
19	0.894107	19	0.917233
20	0.884480	20	0.909538
21	0.874427	21	0.901298
22	0.863919	22	0.892398
23	0.852904	23	0.882879
24	0.841324	24	0.872709
25	0.829138	25	0.861794
26	0.816269	26	0.850149
27	0.802641	27	0.837632
28	0.788166	28	0.824169
29	0.772719	29	0.809639
30	0.756101	30	0.793828
31	0.738148	31	0.776627
32	0.718582	32	0.757696
33	0.697008	33	0.736540
34	0.672980	34	0.712833
35	0.645808	35	0.685960
36	0.614503	36	0.654861
37	0.577210	37	0.617421
38	0.530444	38	0.569983
39	0.465796	39	0.503936
40	0.351763	40	0.386834

Table 5: Survival Probabilities for 5 Pursuers vs. 4 Evaders

5 Pursuers vs. 4 Evaders with GCN - Hot Starts 5 Pursuers vs. 4 Evaders with Random Starts

Time	Survival Probability	Time	Survival Probability
0	1.000000	0	1.000000
1	0.999006	1	0.997989
2	0.997873	2	0.994624
3	0.996323	3	0.991179
4	0.993675	4	0.987612
5	0.989947	5	0.983902
6	0.985225	6	0.980035
7	0.979768	7	0.975990
8	0.973911	8	0.971731
9	0.967684	9	0.967220
10	0.961106	10	0.962384
11	0.954195	11	0.957233
12	0.946975	12	0.951731
13	0.939429	13	0.945845
14	0.931536	14	0.939566
15	0.923328	15	0.932819
16	0.914780	16	0.925681
17	0.905861	17	0.918108
18	0.896497	18	0.910039
19	0.886741	19	0.901412
20	0.876601	20	0.892220
21	0.866038	21	0.882472
22	0.854959	22	0.872068
23	0.843363	23	0.861005
24	0.831203	24	0.849204
25	0.818387	25	0.836610
26	0.804829	26	0.823179
27	0.790460	27	0.808853
28	0.775195	28	0.793553
29	0.758881	29	0.777094
30	0.741385	30	0.759270
31	0.722554	31	0.739848
32	0.702146	32	0.718577
33	0.679747	33	0.695026
34	0.654883	34	0.668694
35	0.626821	35	0.638804
36	0.594471	36	0.604321
37	0.556039	37	0.563265
38	0.507923	38	0.511989
39	0.441791	39	0.441693
40	0.326174	40	0.319653

Table 6: Survival Probabilities for 4 Pursuers vs. 5 Evaders

4 Pursuers vs. 5 Evaders with GCN -
Hot Starts

Time	Survival Probability
0	1.000000
1	0.999111
2	0.998097
3	0.996702
4	0.994433
5	0.991136
6	0.986983
7	0.982107
8	0.976650
9	0.971018
10	0.964981
11	0.958622
12	0.951959
13	0.945030
14	0.937834
15	0.930331
16	0.922515
17	0.914364
18	0.905882
19	0.897037
20	0.887789
21	0.878097
22	0.867943
23	0.857282
24	0.846027
25	0.834158
26	0.821579
27	0.808211
28	0.793962
29	0.778708
30	0.762245
31	0.744428
32	0.724961
33	0.703507
34	0.679653
35	0.652665
36	0.621402
37	0.584048
38	0.537081
39	0.472121
40	0.357325

4 Pursuers vs. 5 Evaders with Ran-
dom Starts

Time	Survival Probability
0	1.000000
1	0.998219
2	0.995346
3	0.992389
4	0.989340
5	0.986164
6	0.982834
7	0.979360
8	0.975713
9	0.971886
10	0.967864
11	0.963565
12	0.958984
13	0.954065
14	0.948824
15	0.943268
16	0.937344
17	0.931068
18	0.924379
19	0.917233
20	0.909538
21	0.901298
22	0.892398
23	0.882879
24	0.872709
25	0.861794
26	0.850149
27	0.837632
28	0.824169
29	0.809639
30	0.793828
31	0.776627
32	0.757696
33	0.736540
34	0.712833
35	0.685960
36	0.654861
37	0.617386
38	0.570022
39	0.503928
40	0.386160

Table 7: Survival Probabilities for 5 Pursuers vs. 5 Evaders

5 Pursuers vs. 5 Evaders with GCN - Hot Starts 5 Pursuers vs. 5 Evaders with Random Starts

Time	Survival Probability	Time	Survival Probability
0	1.000000	0	1.000000
1	0.999070	1	0.997945
2	0.998004	2	0.994521
3	0.996513	3	0.990998
4	0.993849	4	0.987299
5	0.990070	5	0.983443
6	0.985211	6	0.979421
7	0.979584	7	0.975179
8	0.973390	8	0.970706
9	0.966780	9	0.965971
10	0.959835	10	0.960984
11	0.952516	11	0.955716
12	0.944874	12	0.950118
13	0.936892	13	0.944165
14	0.928621	14	0.937817
15	0.920041	15	0.931047
16	0.911140	16	0.923794
17	0.901895	17	0.916031
18	0.892243	18	0.907752
19	0.882160	19	0.898889
20	0.871633	20	0.889436
21	0.860602	21	0.879392
22	0.849035	22	0.868607
23	0.836887	23	0.857125
24	0.824146	24	0.844874
25	0.810707	25	0.831829
26	0.796522	26	0.817994
27	0.781482	27	0.803284
28	0.765492	28	0.787488
29	0.748426	29	0.770572
30	0.730154	30	0.752280
31	0.710437	31	0.732417
32	0.689038	32	0.710725
33	0.665604	33	0.686770
34	0.639634	34	0.659997
35	0.610420	35	0.629638
36	0.576881	36	0.594463
37	0.537168	37	0.552714
38	0.487778	38	0.500678
39	0.420328	39	0.429612
40	0.303443	40	0.307052

G.2 One vs. Many Supplementary Tables

In this subsection, we see the survival probabilities for the one vs. many games. We see that hot starts had a much smaller impact here, this is because the pursuers have such a large number advantage in comparison to the evader. The advantage given by a hot start has little influence on the evader survival probabilities, as already the pursuers have a large advantage.

Table 8: Survival Probabilities for 2 Pursuers vs. 1 Evader

2 Pursuers vs. 1 Evader with GCN - Hot Starts 2 Pursuers vs. 1 Evader with Random Starts

Time	Survival Probability	Time	Survival Probability
0	1.000000	0	1.000000
1	0.991712	1	0.993463
2	0.983283	2	0.986801
3	0.974511	3	0.979894
4	0.965517	4	0.972637
5	0.956183	5	0.965018
6	0.946949	6	0.957064
7	0.937354	7	0.948788
8	0.927597	8	0.940033
9	0.917843	9	0.930603
10	0.907954	10	0.920607
11	0.898163	11	0.910143
12	0.888222	12	0.899190
13	0.878467	13	0.887629
14	0.868690	14	0.875333
15	0.859034	15	0.862641
16	0.849344	16	0.849322
17	0.839081	17	0.835078
18	0.827845	18	0.820283
19	0.814938	19	0.804828
20	0.799280	20	0.788348
21	0.781656	21	0.771300
22	0.761806	22	0.753276
23	0.739184	23	0.733586
24	0.714617	24	0.712938
25	0.689271	25	0.691772
26	0.663882	26	0.669866
27	0.639366	27	0.646732
28	0.616202	28	0.623076
29	0.594326	29	0.597842
30	0.573309	30	0.570885
31	0.551781	31	0.542512
32	0.528637	32	0.512252
33	0.502271	33	0.480236
34	0.470700	34	0.445796
35	0.431475	35	0.407903
36	0.385005	36	0.366012
37	0.330575	37	0.318201
38	0.266333	38	0.263206
39	0.191560	39	0.195693
40	0.089746	40	0.094324

Table 9: Survival Probabilities for 3 Pursuers vs. 1 Evader

3 Pursuers vs. 1 Evader with GCN - Hot Starts 3 Pursuers vs. 1 Evader with Random Starts

Time	Survival Probability	Time	Survival Probability
0	1.000000	0	1.000000
1	0.991542	1	0.993342
2	0.982940	2	0.986558
3	0.973990	3	0.979592
4	0.964908	4	0.972267
5	0.955572	5	0.964462
6	0.946125	6	0.956222
7	0.936274	7	0.947448
8	0.926023	8	0.938145
9	0.915364	9	0.928314
10	0.904409	10	0.917903
11	0.893104	11	0.906899
12	0.881432	12	0.895380
13	0.869092	13	0.882983
14	0.856554	14	0.869967
15	0.843397	15	0.856226
16	0.829981	16	0.841693
17	0.815929	17	0.826461
18	0.801278	18	0.810387
19	0.785568	19	0.793565
20	0.768335	20	0.775779
21	0.749677	21	0.757549
22	0.729725	22	0.738543
23	0.708374	23	0.718767
24	0.685581	24	0.698388
25	0.661486	25	0.676883
26	0.636702	26	0.653929
27	0.611265	27	0.629687
28	0.585529	28	0.603854
29	0.559782	29	0.576597
30	0.533947	30	0.548011
31	0.507748	31	0.517688
32	0.480724	32	0.485342
33	0.452001	33	0.451186
34	0.421050	34	0.415005
35	0.386524	35	0.376779
36	0.346016	36	0.335710
37	0.298554	37	0.290361
38	0.242127	38	0.237999
39	0.173000	39	0.173898
40	0.073006	40	0.078950

Table 10: Survival Probabilities for 4 Pursuers vs. 1 Evader

4 Pursuers vs. 1 Evader with GCN - Hot Starts 4 Pursuers vs. 1 Evader with Random Starts

Time	Survival Probability	Time	Survival Probability
0	1.000000	0	1.000000
1	0.991600	1	0.993475
2	0.983057	2	0.986826
3	0.974306	3	0.979912
4	0.965630	4	0.972702
5	0.956510	5	0.965028
6	0.947177	6	0.956867
7	0.937420	7	0.948178
8	0.927343	8	0.938947
9	0.916736	9	0.928949
10	0.905824	10	0.918191
11	0.894433	11	0.906515
12	0.882366	12	0.895996
13	0.869911	13	0.882618
14	0.856854	14	0.868438
15	0.843144	15	0.851609
16	0.828870	16	0.836041
17	0.813872	17	0.819477
18	0.797834	18	0.801867
19	0.780508	19	0.783452
20	0.761804	20	0.763779
21	0.741454	21	0.742971
22	0.719728	22	0.721122
23	0.696346	23	0.698146
24	0.671677	24	0.674348
25	0.645555	25	0.649481
26	0.618582	26	0.623199
27	0.590867	27	0.595733
28	0.562506	28	0.567184
29	0.533408	29	0.537419
30	0.503428	30	0.506212
31	0.472656	31	0.473409
32	0.440436	32	0.438982
33	0.406234	33	0.402917
34	0.369310	34	0.364424
35	0.329163	35	0.323153
36	0.284956	36	0.291866
37	0.236425	37	0.244088
38	0.181495	38	0.189949
39	0.117632	39	0.125860
40	0.034466	40	0.035190

Table 11: Survival Probabilities for 5 Pursuers vs. 1 Evader

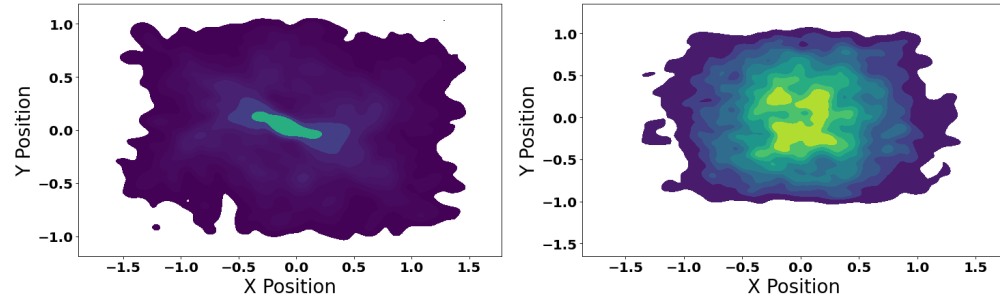
5 Pursuers vs. 1 Evader with GCN - Hot Starts 5 Pursuers vs. 1 Evader with Random Starts

Time	Survival Probability	Time	Survival Probability
0	1.000000	0	1.000000
1	0.991545	1	0.993080
2	0.982947	2	0.986032
3	0.974100	3	0.978725
4	0.965133	4	0.971133
5	0.955943	5	0.963078
6	0.946558	6	0.954553
7	0.936875	7	0.945502
8	0.926854	8	0.938145
9	0.916600	9	0.928314
10	0.905950	10	0.917903
11	0.894928	11	0.906899
12	0.883454	12	0.895380
13	0.871427	13	0.882983
14	0.858801	14	0.869967
15	0.845529	15	0.856226
16	0.831392	16	0.841693
17	0.816614	17	0.826461
18	0.801055	18	0.810387
19	0.784466	19	0.793565
20	0.766864	20	0.775779
21	0.747930	21	0.757549
22	0.727870	22	0.738543
23	0.706147	23	0.718767
24	0.683110	24	0.698388
25	0.658689	25	0.676883
26	0.633246	26	0.653929
27	0.607021	27	0.629687
28	0.579705	28	0.603854
29	0.551502	29	0.576597
30	0.522533	30	0.548011
31	0.492414	31	0.517688
32	0.460528	32	0.485342
33	0.426760	33	0.451186
34	0.390509	34	0.415005
35	0.351276	35	0.376779
36	0.308013	36	0.335710
37	0.259501	37	0.290361
38	0.204193	38	0.237999
39	0.138545	39	0.173898
40	0.049294	40	0.040376

G.3 One vs. Many Pursuers Movement Analysis

In this section, we show conditional density plots of containment, and we also show heatmaps of pursuers movement, the brighter the color the more active the pursuers were.

G.3.1 1 Evader vs. 2 Pursuers Movement Analysis

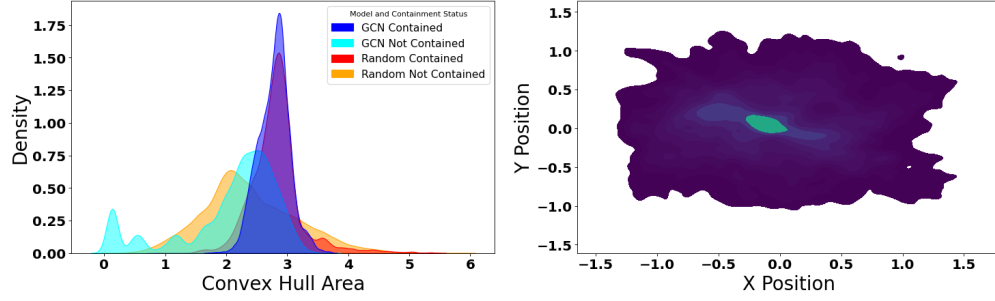


(a) Heat map showing movement patterns with hot start strategies for 2 pursuers vs. 1 evader.

(b) Heat map showing movement patterns with random start strategies for 2 pursuers vs. 1 evader.

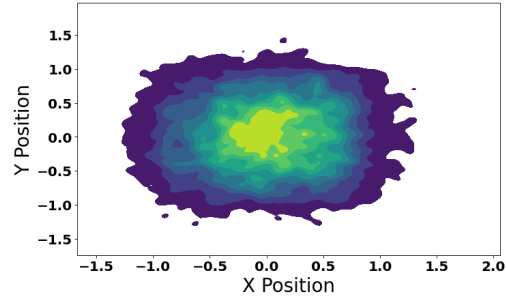
Figure 4: Movement analysis for 2 Pursuers vs. 1 Evader games.

G.3.2 1 Evader vs. 3 Pursuers Movement Analysis



(a) Conditional Density Plot showing containment metrics for 3 pursuers against 1 evader.

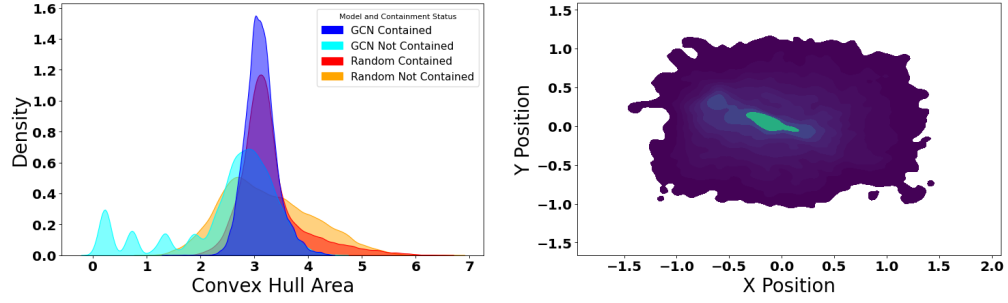
(b) Heat map showing movement patterns with hot start strategies for 3 pursuers vs. 1 evader.



(c) Heat map showing movement patterns with random start strategies for 3 pursuers vs. 1 evader.

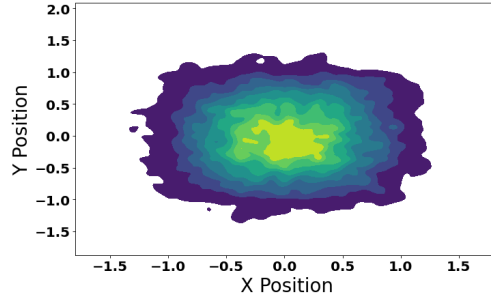
Figure 5: Movement analysis for 3 Pursuers vs. 1 Evader games.

G.3.3 1 Evader vs. 5 Pursuers Movement Analysis



(a) Conditional Density Plot showing containment metrics for 5 pursuers against 1 evader.

(b) Heat map showing movement patterns with hot start strategies for 5 pursuers vs. 1 evader.



(c) Heat map showing movement patterns with random start strategies for 5 pursuers vs. 1 evader.

Figure 6: Movement analysis for 5 Pursuers vs. 1 Evader games.

G.4 Many vs. Many Pursuers Movement Analysis

G.4.1 4 Pursuers vs. 3 Evaders Movement Analysis

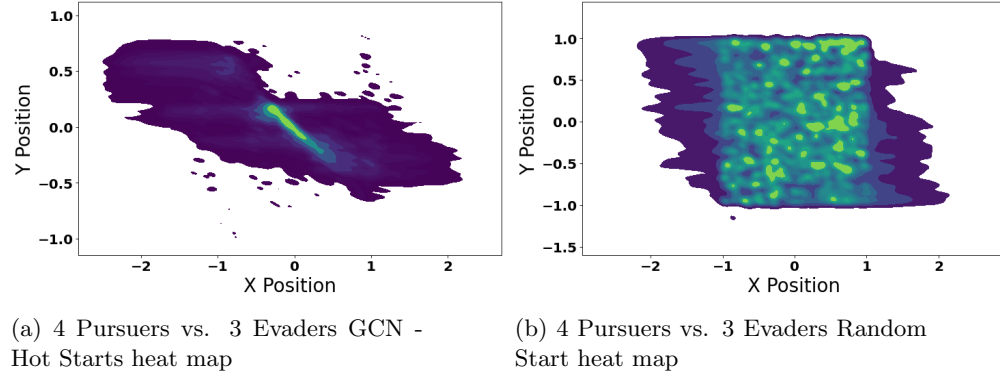


Figure 7: Heat Maps for 4 Pursuers vs. 3 Evader Games. The brighter the color, the more activity in that area of movement.

G.4.2 4 Pursuers vs. 4 Evaders Movement Analysis

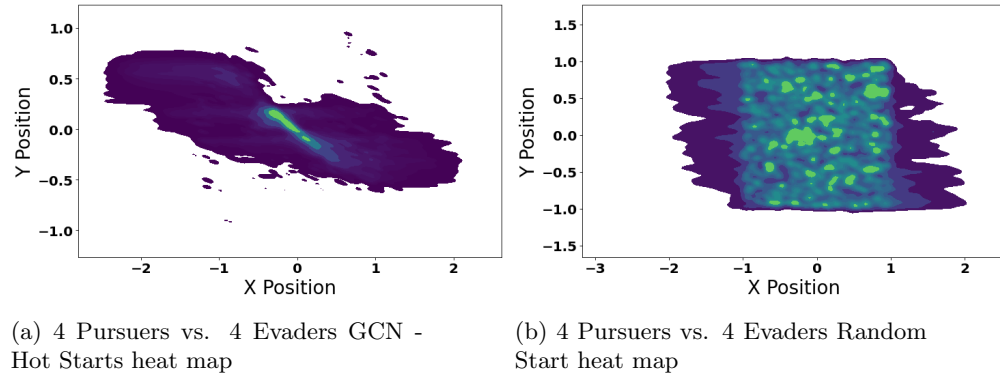
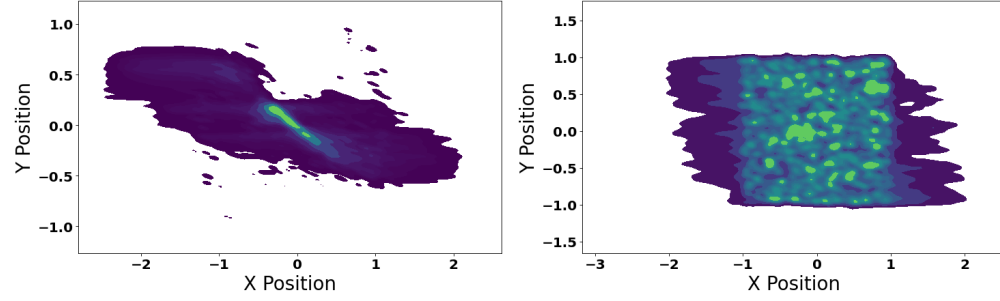


Figure 8: Heat Maps for 4 Pursuers vs. 4 Evaders Games. The brighter the color, the more activity in that area of movement.

G.4.3 4 Pursuers vs. 5 Evaders Movement Analysis

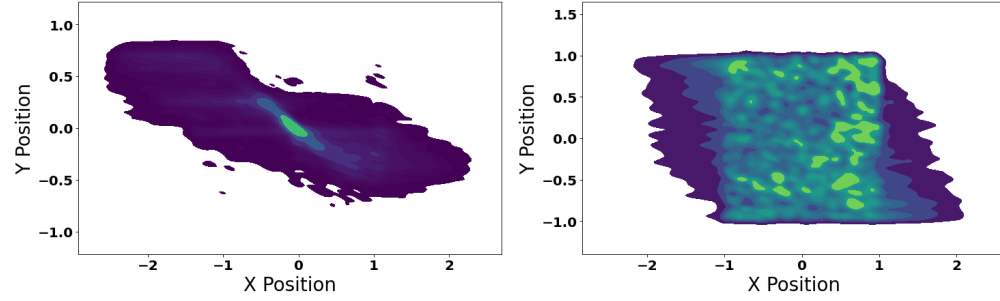


(a) 4 Pursuers vs. 5 Evaders GCN - Hot Starts heat map

(b) 4 Pursuers vs. 5 Evaders Random Start heat map

Figure 9: Heat Maps for 4 Pursuers vs. 5 Evaders Games. The brighter the color, the more activity in that area of movement.

G.4.4 5 Pursuers vs. 2 Evaders Movement Analysis

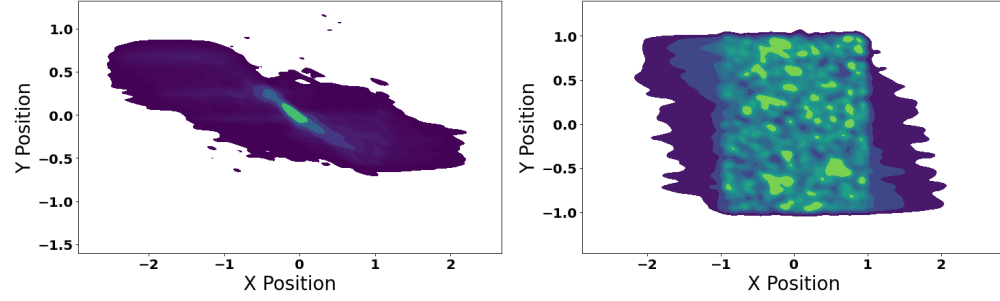


(a) 5 Pursuers vs. 2 Evaders GCN - Hot Starts heat map

(b) 5 Pursuers vs. 2 Evaders Random Start heat map

Figure 10: Heat Maps for 5 Pursuers vs. 2 Evaders Games. The brighter the color, the more activity in that area of movement.

G.4.5 5 Pursuers vs. 3 Evaders Movement Analysis

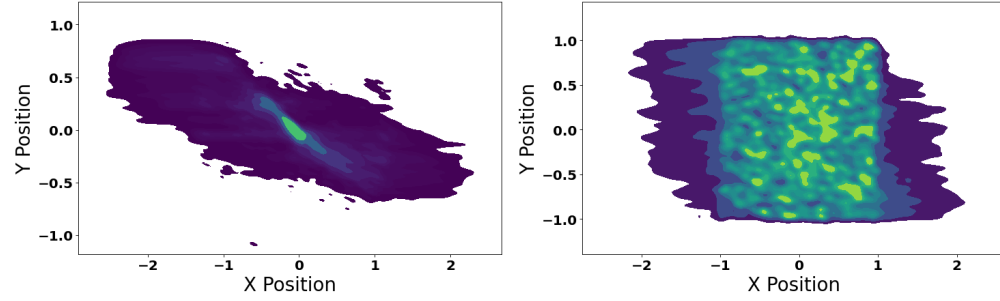


(a) 5 Pursuers vs. 3 Evaders GCN - Hot Starts heat map

(b) 5 Pursuers vs. 3 Evaders Random Start heat map

Figure 11: Heat Maps for 5 Pursuers vs. 3 Evaders Games. The brighter the color, the more activity in that area of movement.

G.4.6 5 Pursuers vs. 4 Evaders Movement Analysis

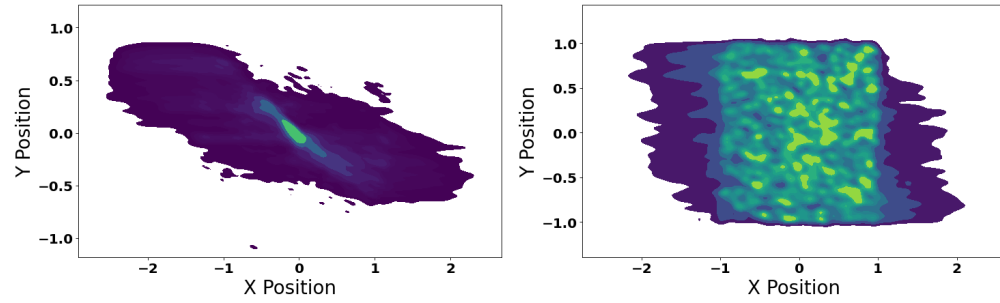


(a) 5 Pursuers vs. 4 Evaders GCN - Hot Starts heat map

(b) 5 Pursuers vs. 4 Evaders Random Start heat map

Figure 12: Heat Maps for 5 Pursuers vs. 4 Evaders Games. The brighter the color, the more activity in that area of movement.

G.4.7 5 Pursuers vs. 5 Evaders Movement Analysis



(a) 5 Pursuers vs. 5 Evaders GCN - Hot Starts heat map

(b) 5 Pursuers vs. 5 Evaders Random Start heat map

Figure 13: Heat Maps for 5 Pursuers vs. 5 Evaders Games. The brighter the color, the more activity in that area of movement.

References

- [1] J. Blank and K. Deb. pymoo: Multi-objective optimization in python. *IEEE Access*, 8:89497–89509, 2020.
- [2] Rufus Isaacs. *Differential games: a mathematical theory with applications to warfare and pursuit, control and optimization*. Courier Corporation, 1999.
- [3] Edward L Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958.
- [4] Ian M. Mitchell, Alexandre M. Bayen, and Claire J. Tomlin. A time-dependent hamilton–jacobi formulation of reachable sets for continuous dynamic games. *IEEE Transactions on Automatic Control*, 50(7):947–957, 2005.
- [5] Alex Von Moll. A few words about differential games. Unpublished manuscript, personal communication, 2024.
- [6] Stanley Osher and Ronald Fedkiw. *Level Set Methods and Dynamic Implicit Surfaces*, volume 153 of *Applied Mathematical Sciences*. Springer, 2003.
- [7] Jason T Rich, J Gail Neely, Randal C Paniello, Courtney CJ Voelker, Brian Nussenbaum, and Eric W Wang. A practical guide to understanding kaplan-meier curves. *Otolaryngology—Head and Neck Surgery*, 143(3):331–336, 2010.